

CS301P FINAL TERM BY IQRA AZHAR

CS301P

Final term file

2024

Solved by

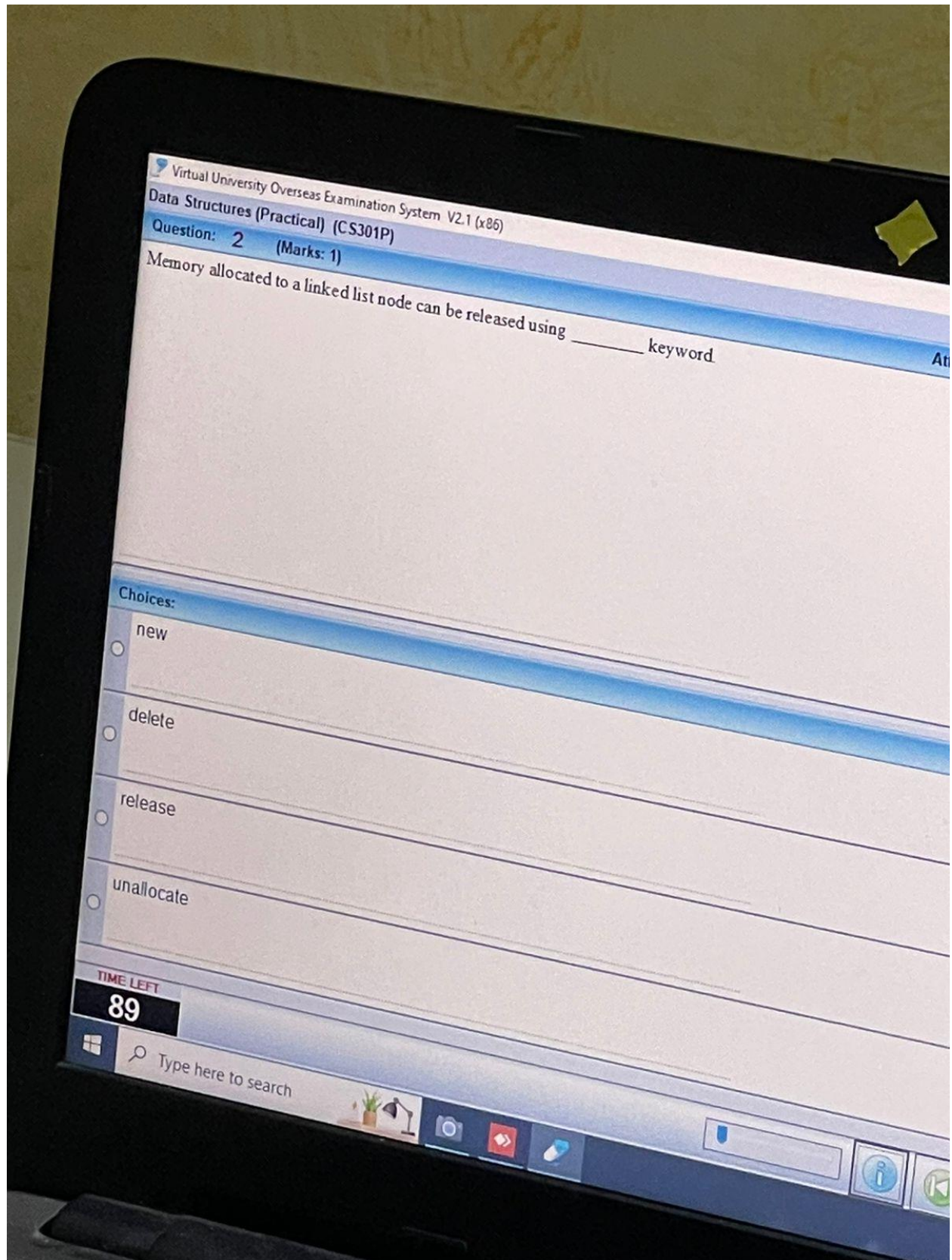
 **Iqra Azhar** 

YTUBE

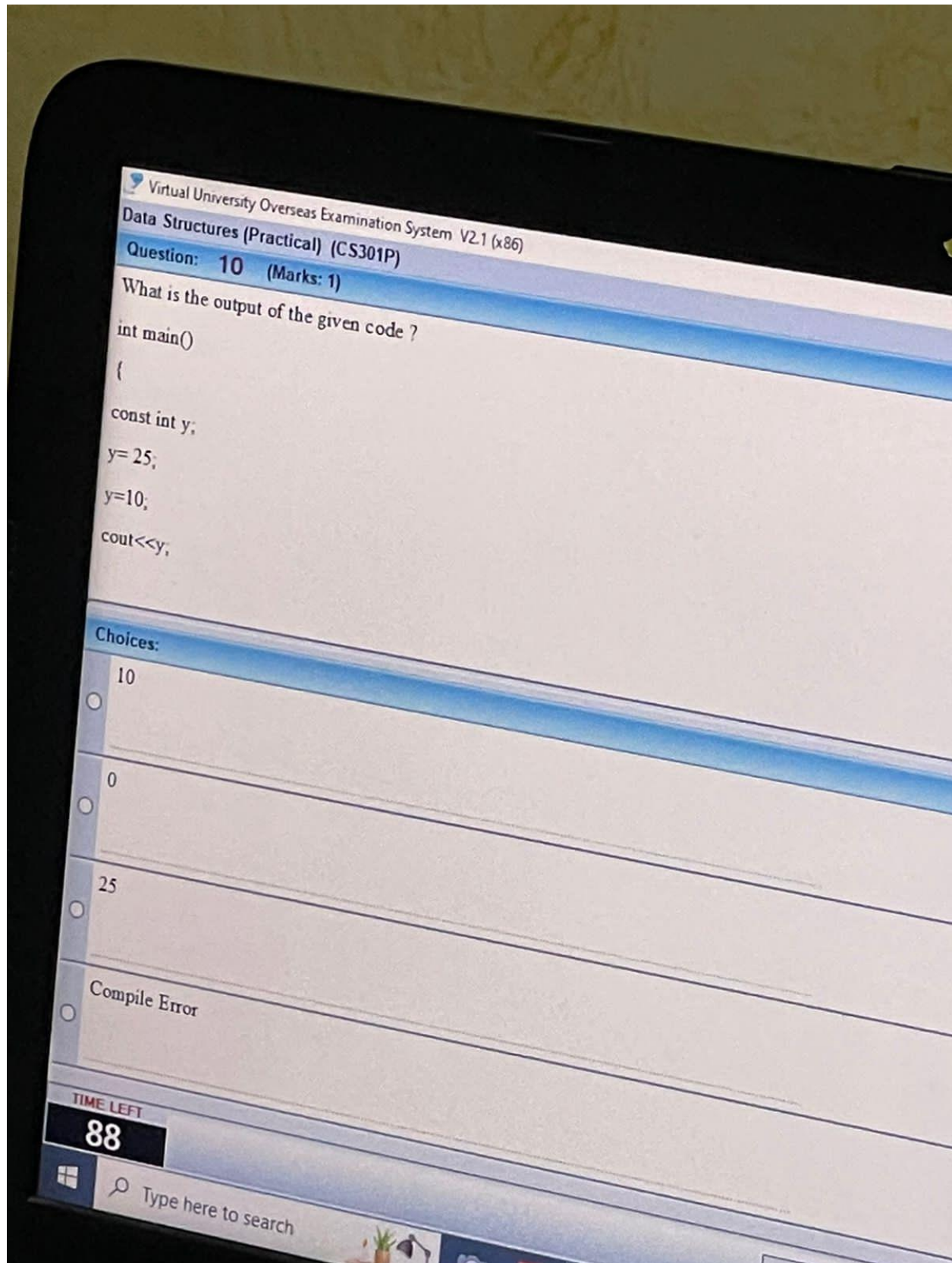
**Assignment||Quiz||Gdb||Project Seminar
||online math classes||online book shop
||Lms Handling Services available**

CONTACT US **03235429064**

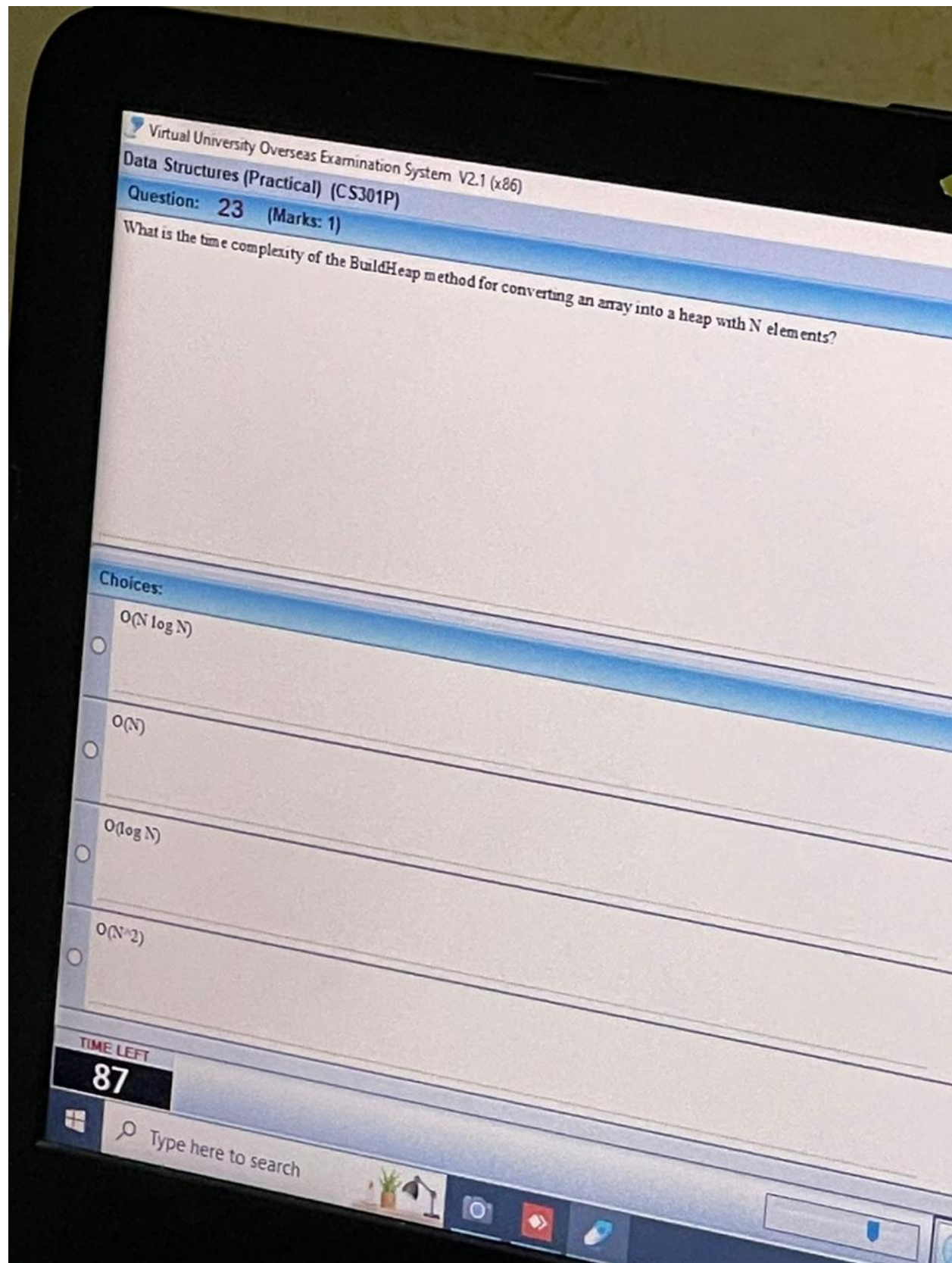
CS301P FINAL TERM BY IQRA AZHAR



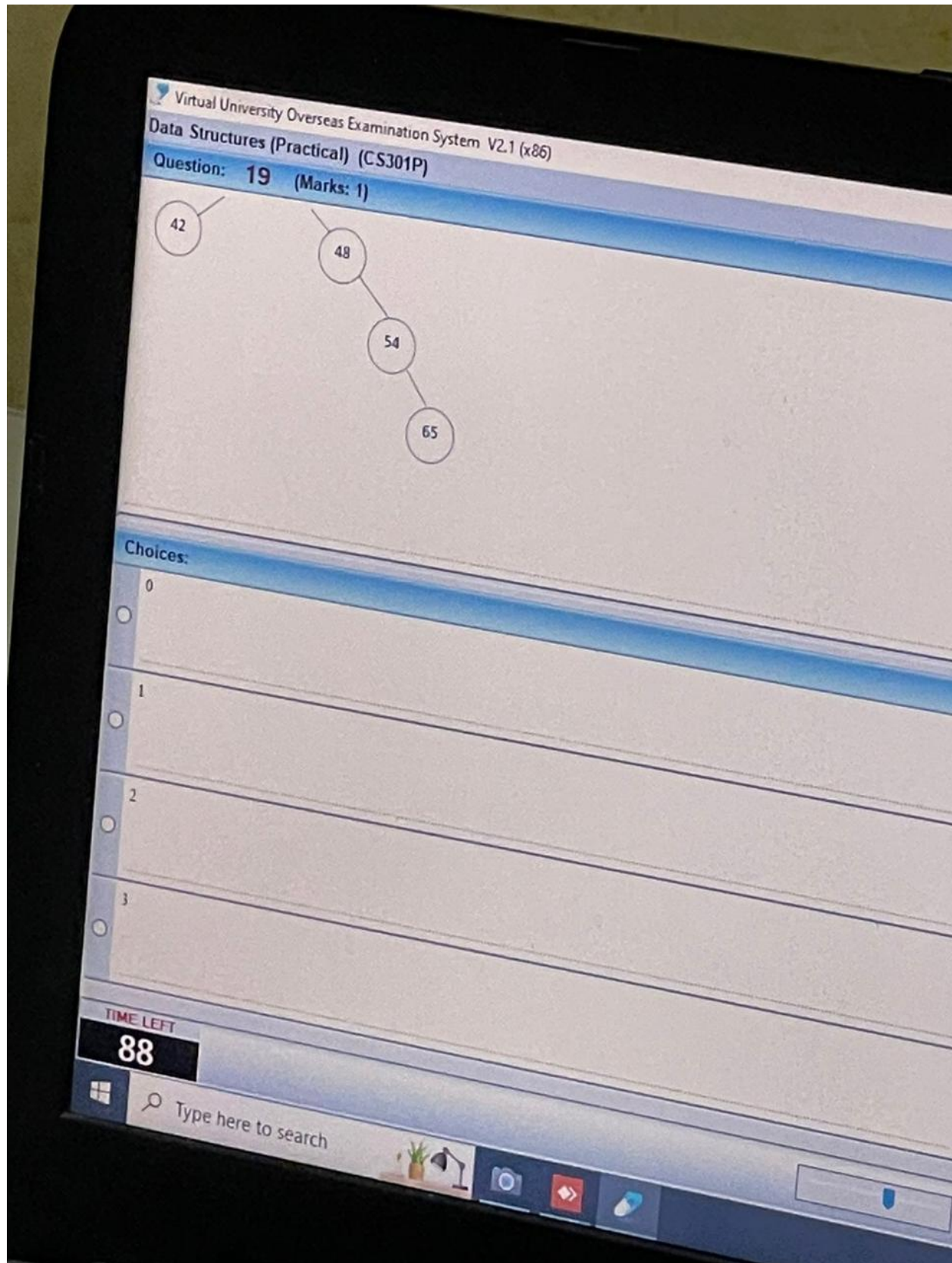
CS301P FINAL TERM BY IQRA AZHAR



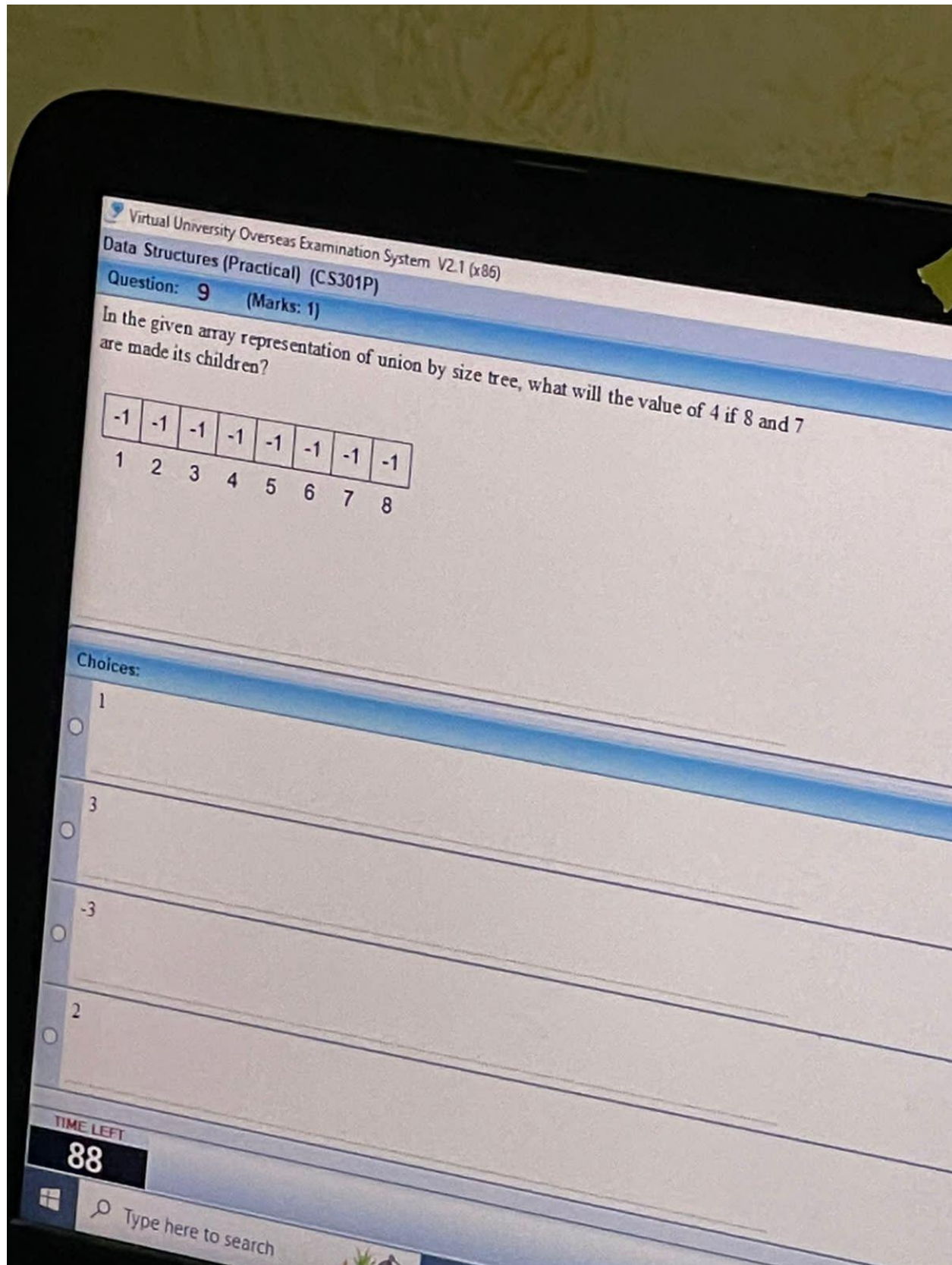
CS301P FINAL TERM BY IQRA AZHAR



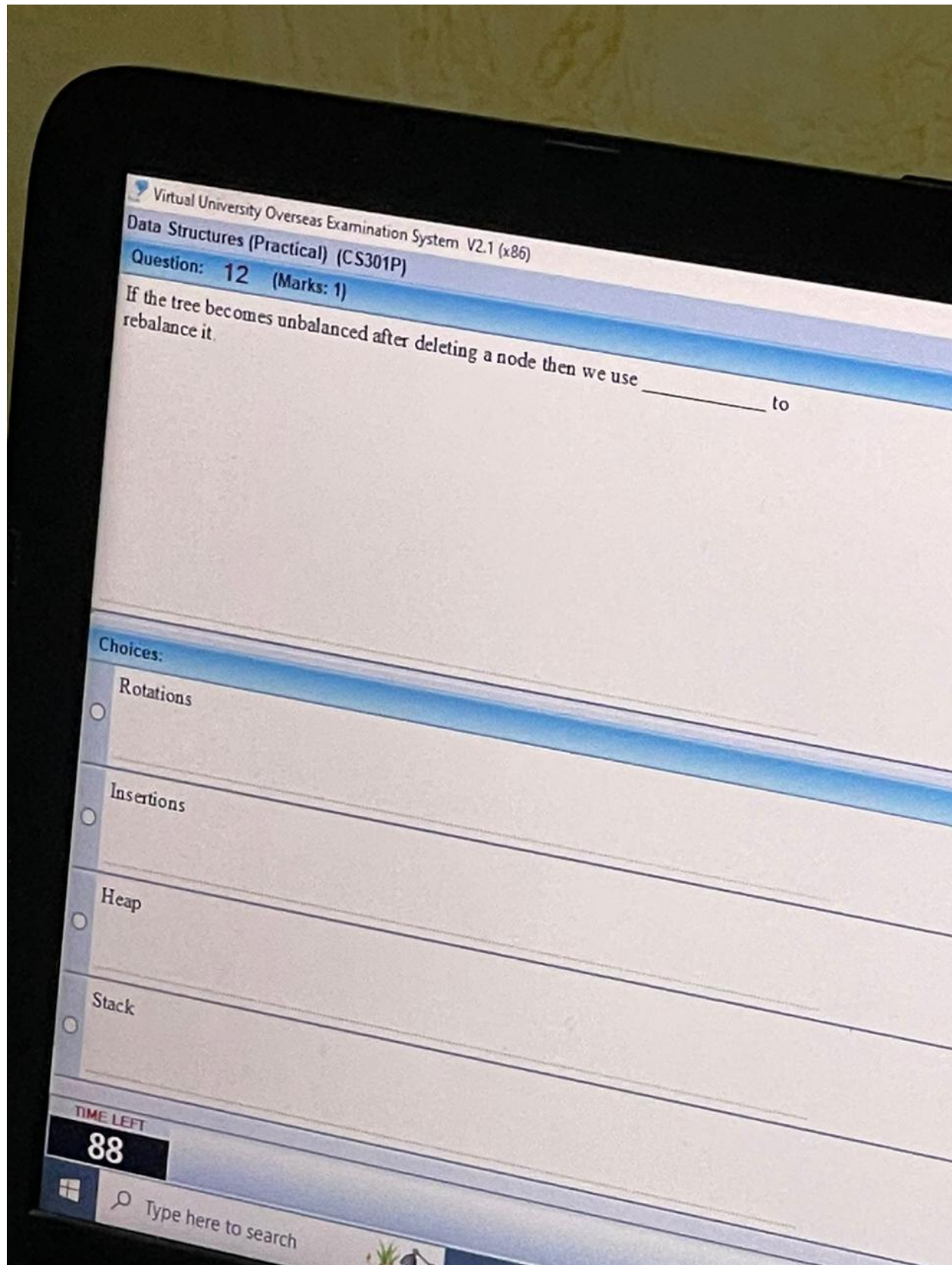
CS301P FINAL TERM BY IQRA AZHAR



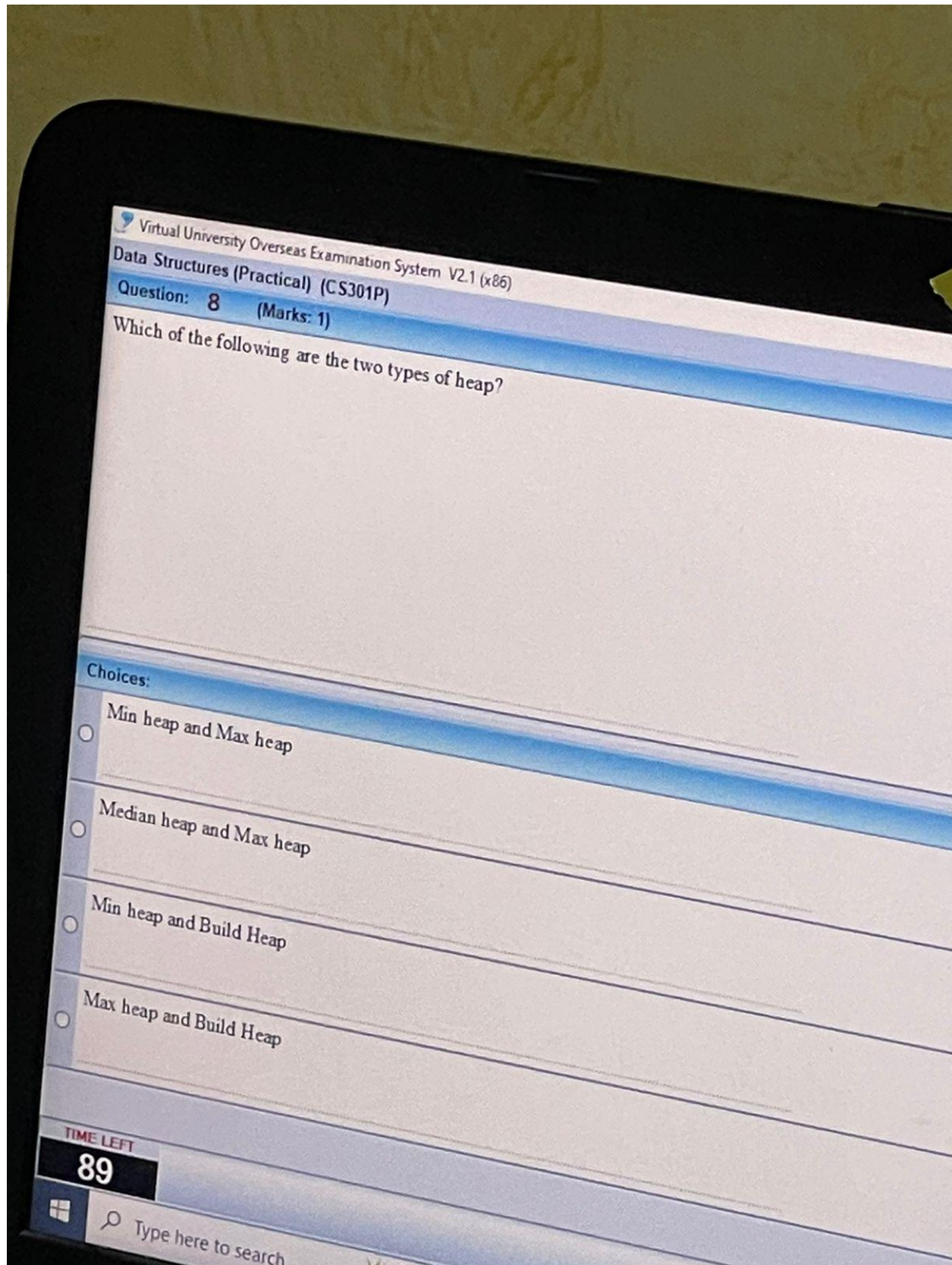
CS301P FINAL TERM BY IQRA AZHAR



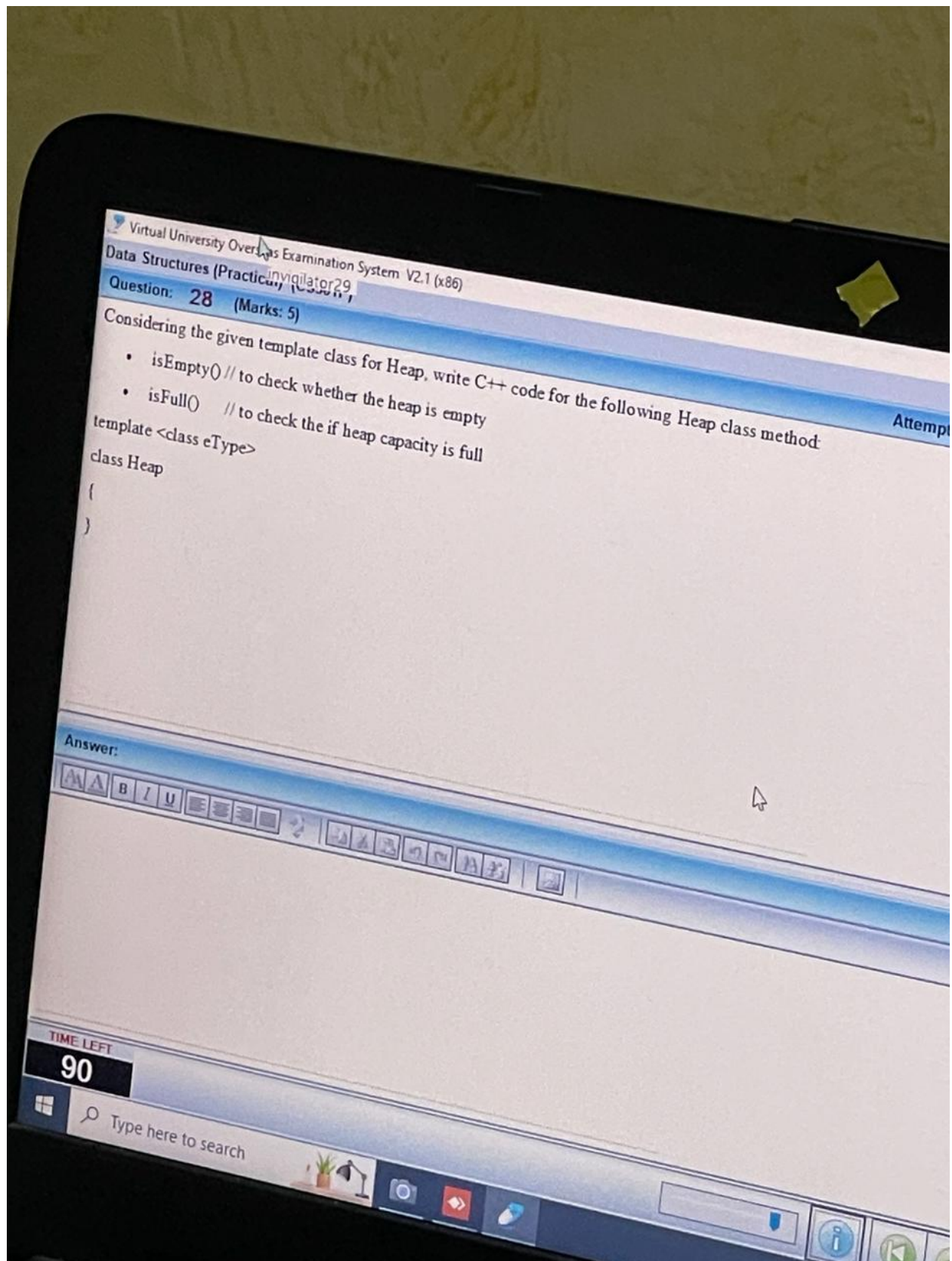
CS301P FINAL TERM BY IQRA AZHAR



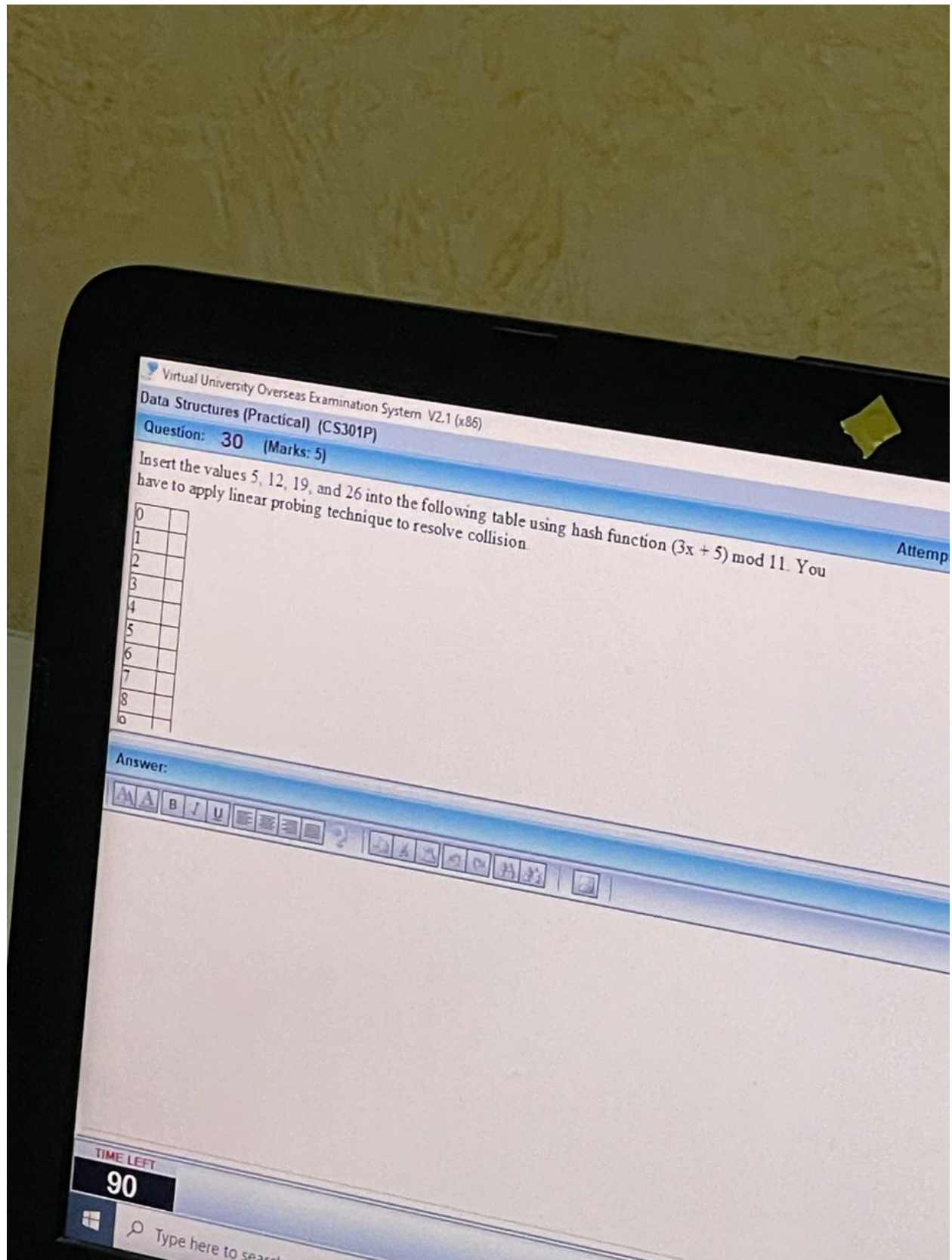
CS301P FINAL TERM BY IQRA AZHAR



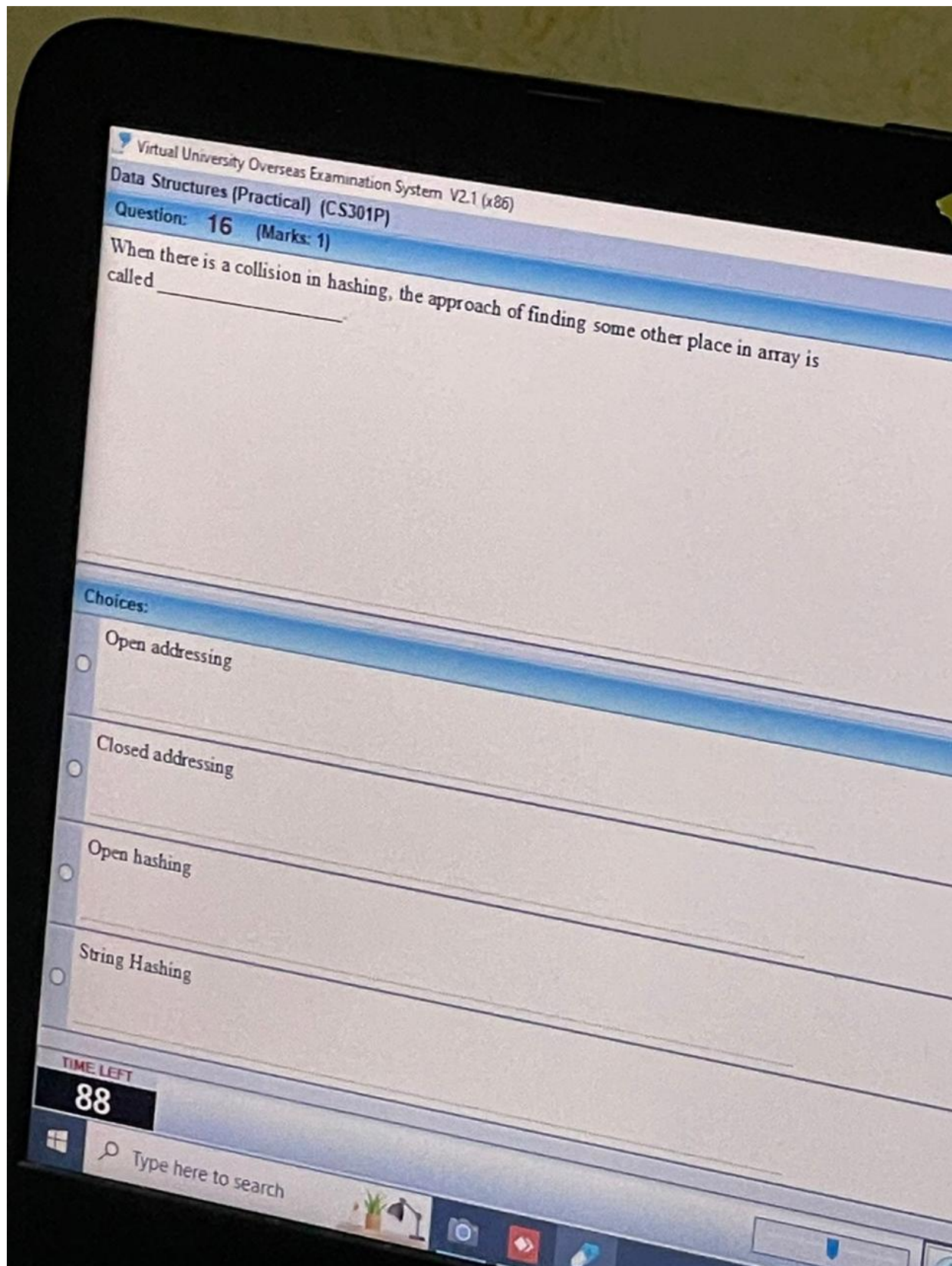
CS301P FINAL TERM BY IQRA AZHAR



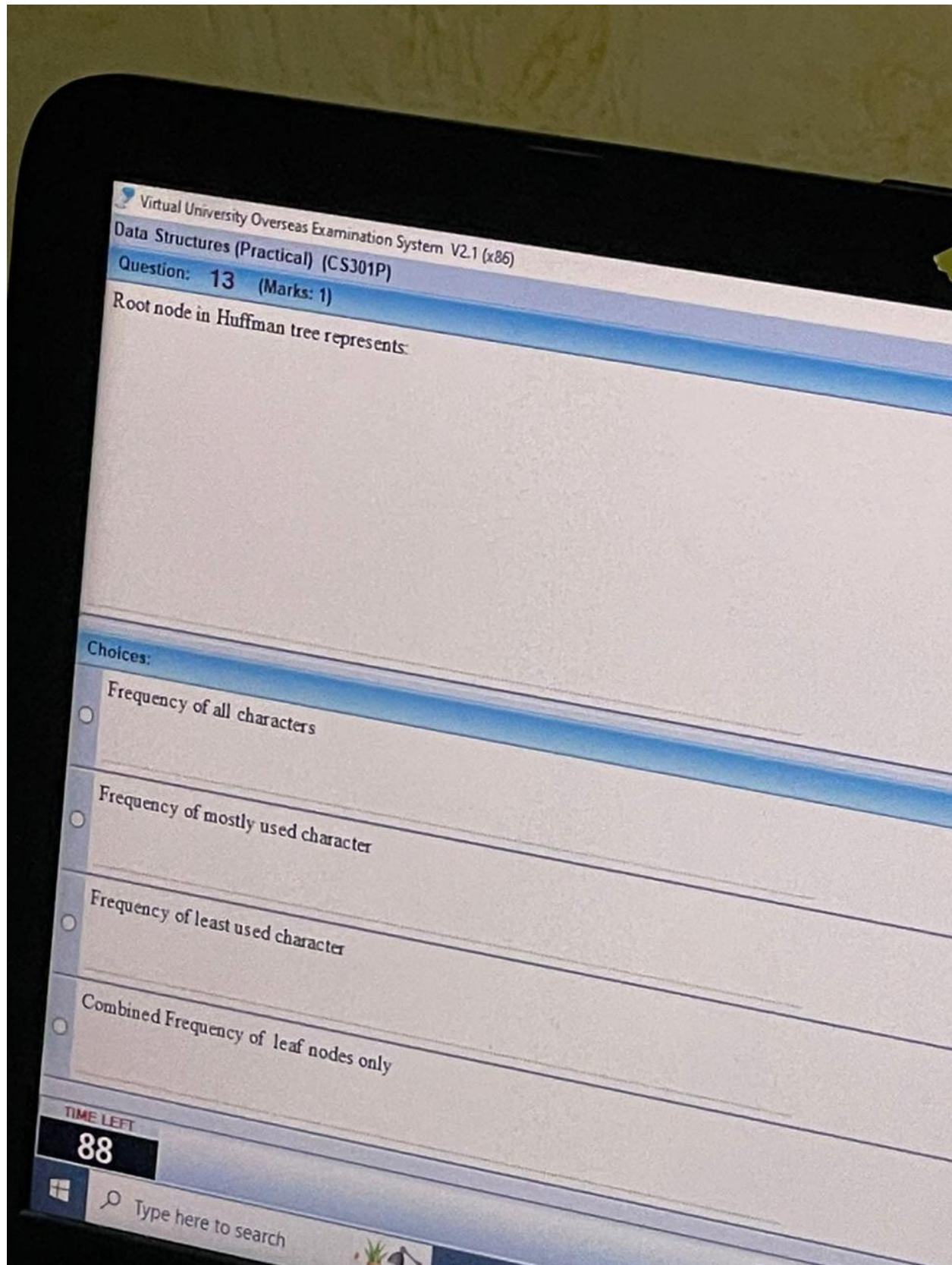
CS301P FINAL TERM BY IQRA AZHAR



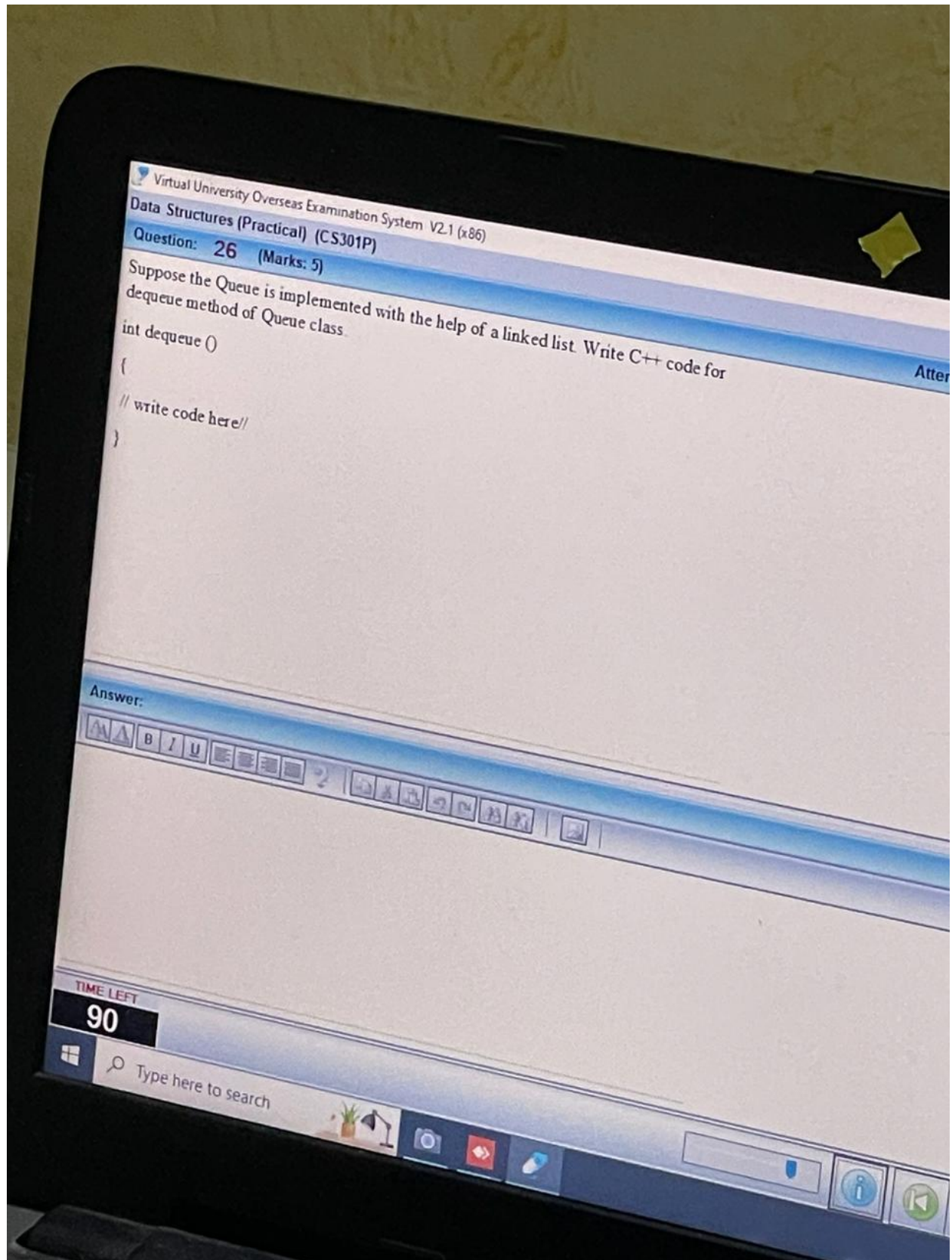
CS301P FINAL TERM BY IQRA AZHAR



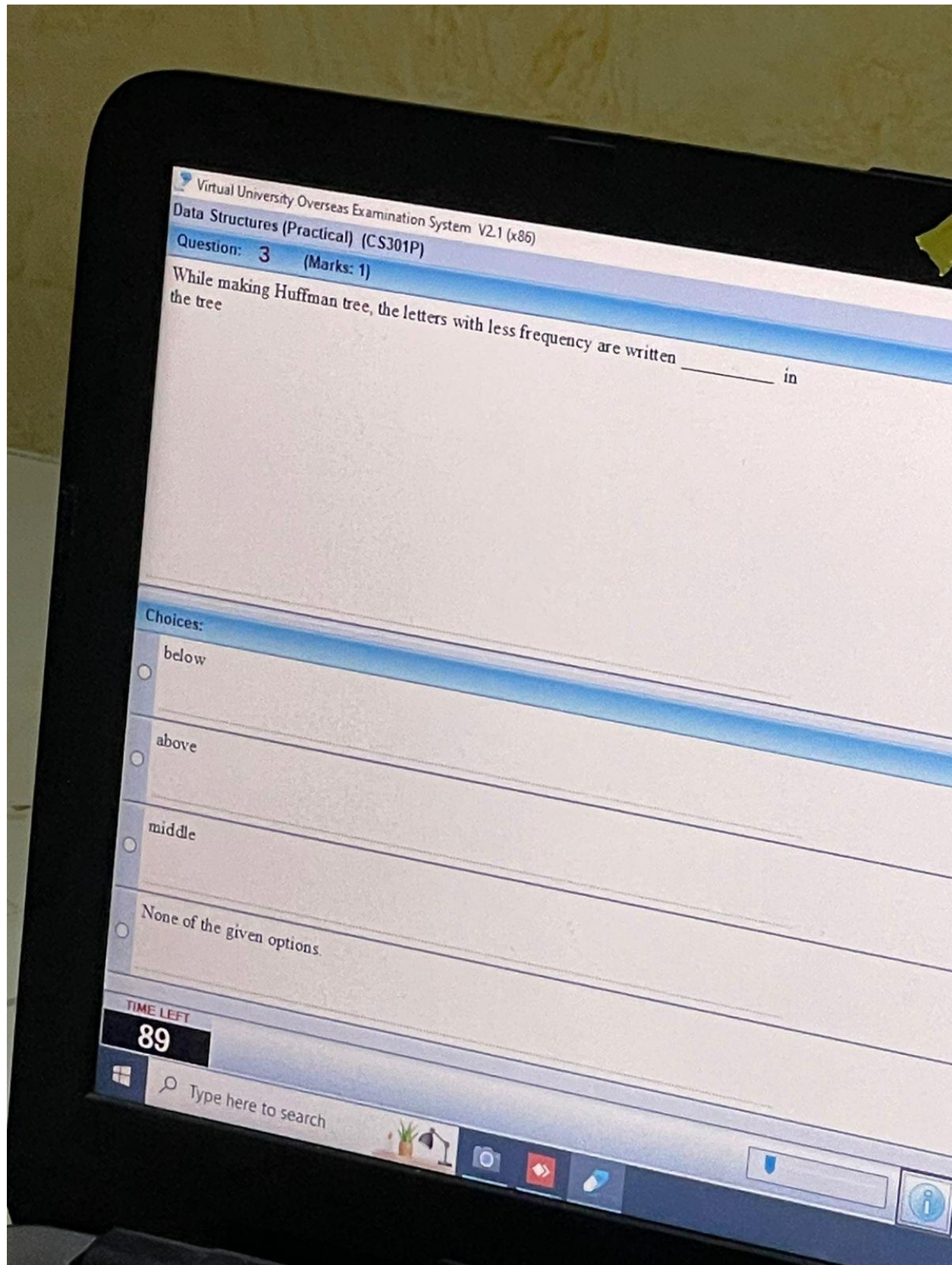
CS301P FINAL TERM BY IQRA AZHAR



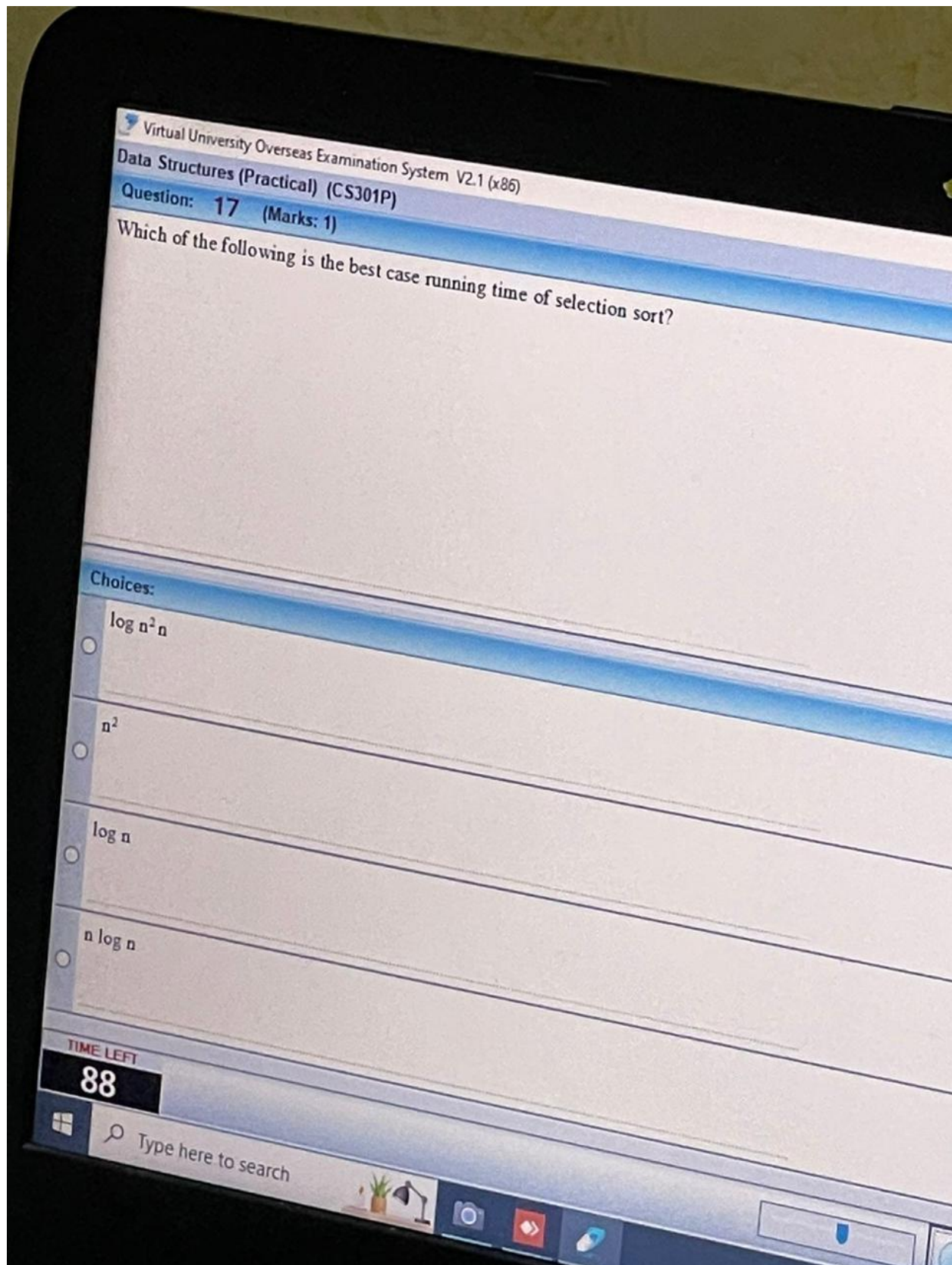
CS301P FINAL TERM BY IQRA AZHAR



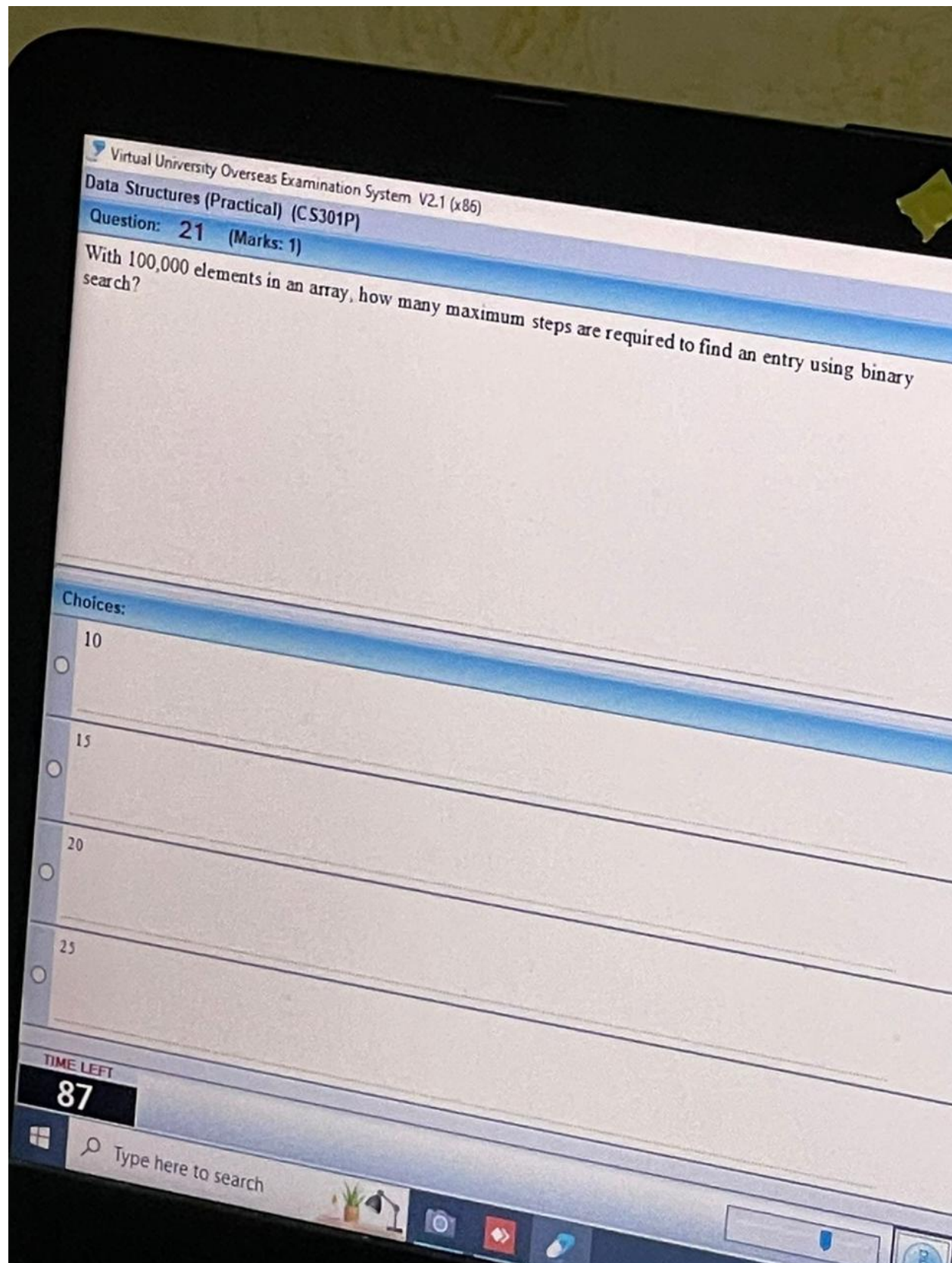
CS301P FINAL TERM BY IQRA AZHAR



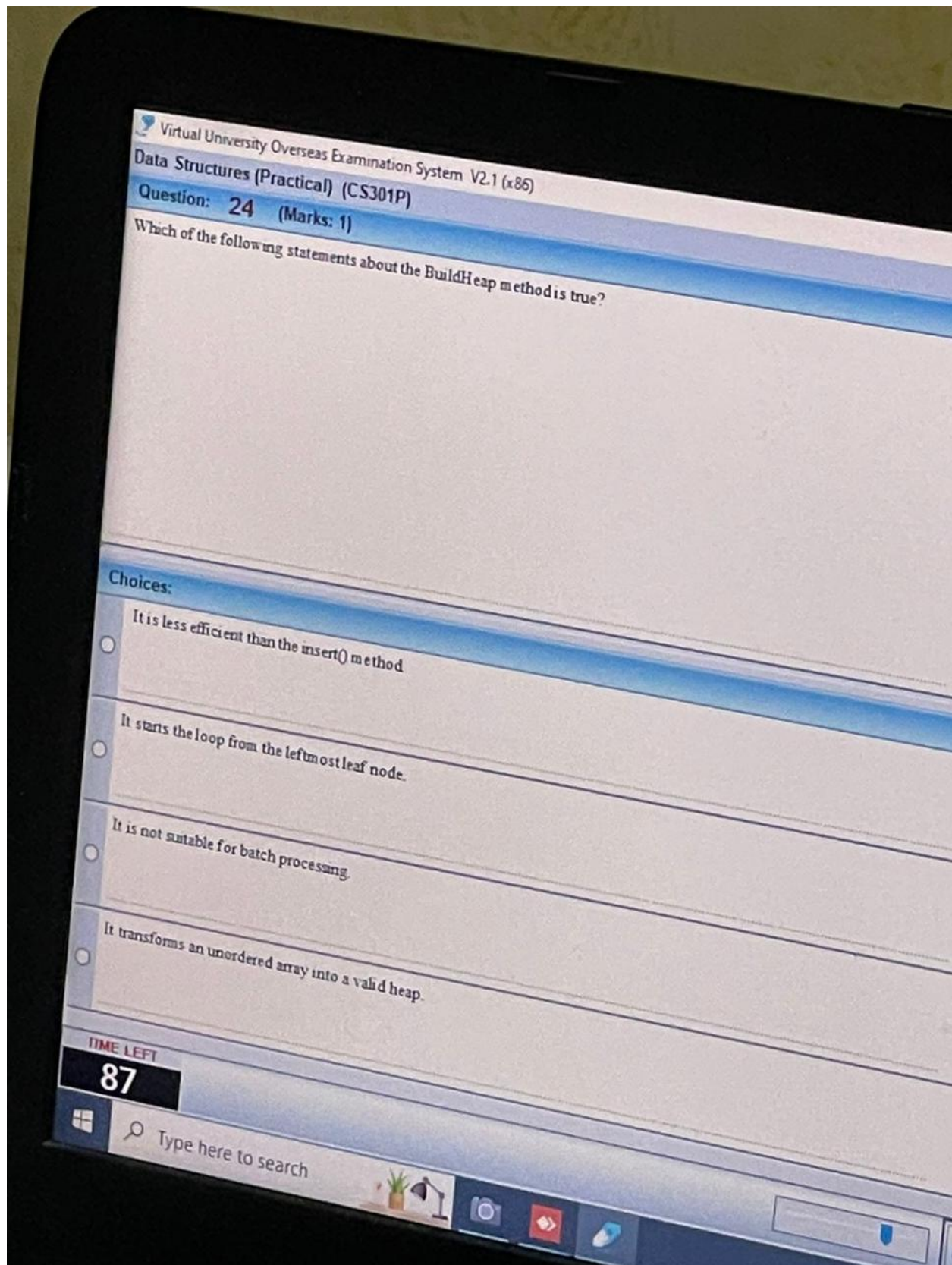
CS301P FINAL TERM BY IQRA AZHAR



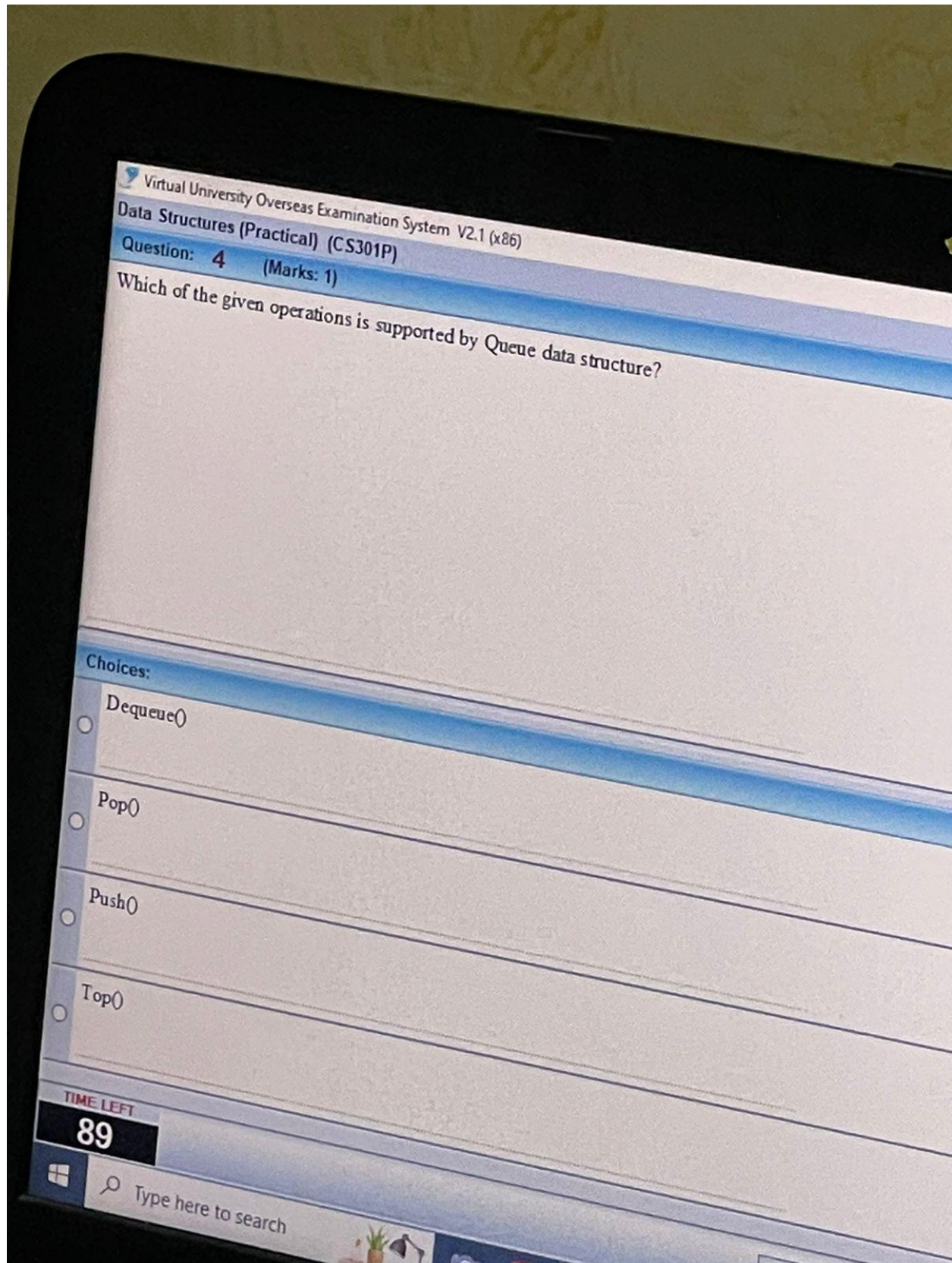
CS301P FINAL TERM BY IQRA AZHAR



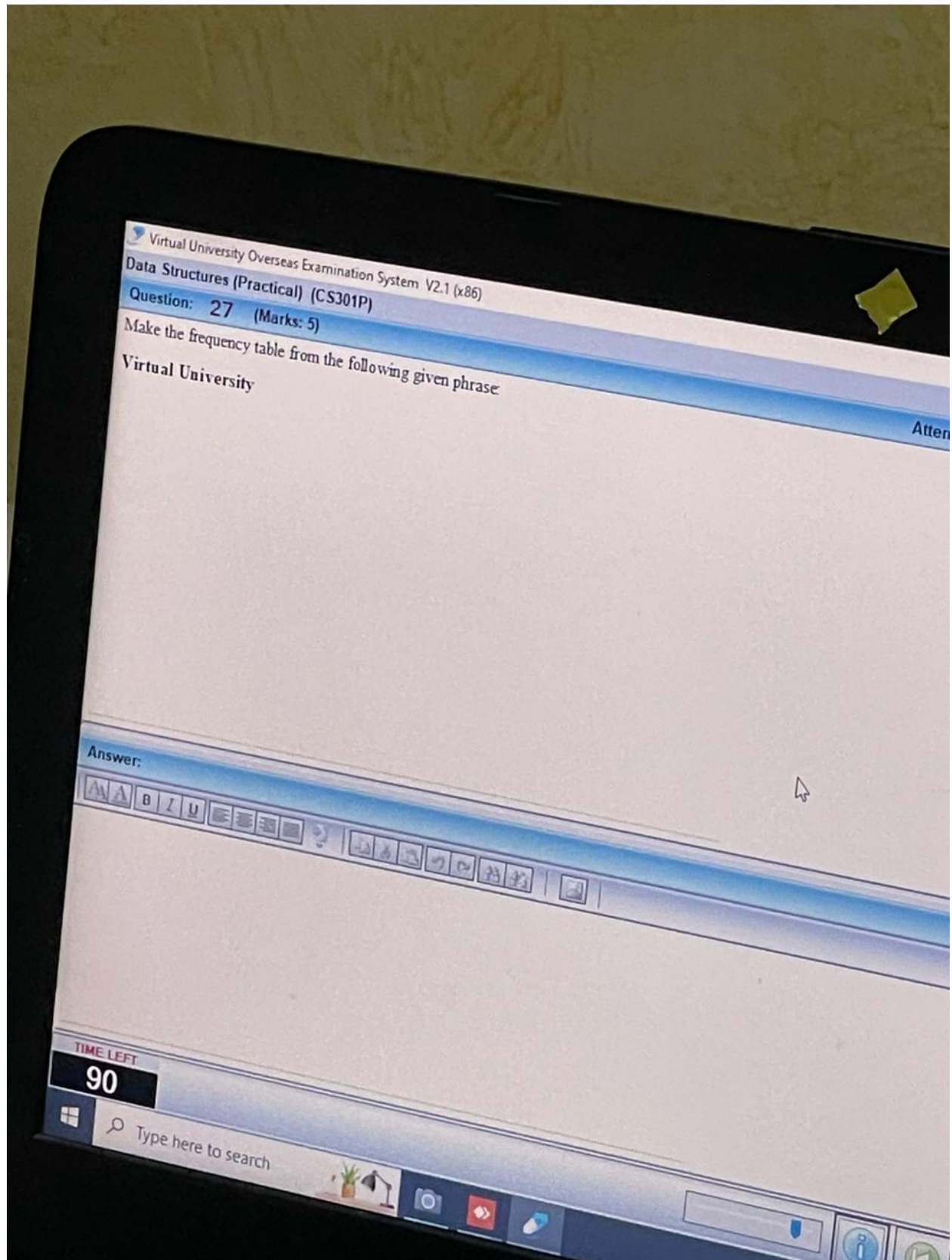
CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR

Virtual University Overseas Examination System V2.1 (x86)

Data Structures (Practical) (CS301P)

Question: 6 (Marks: 1)

Which rotation will be made if node 3 is inserted in the given tree?

```
graph TD; 6((6)) --- 5((5)); 6 --- 9((9)); 5 --- 4((4));
```

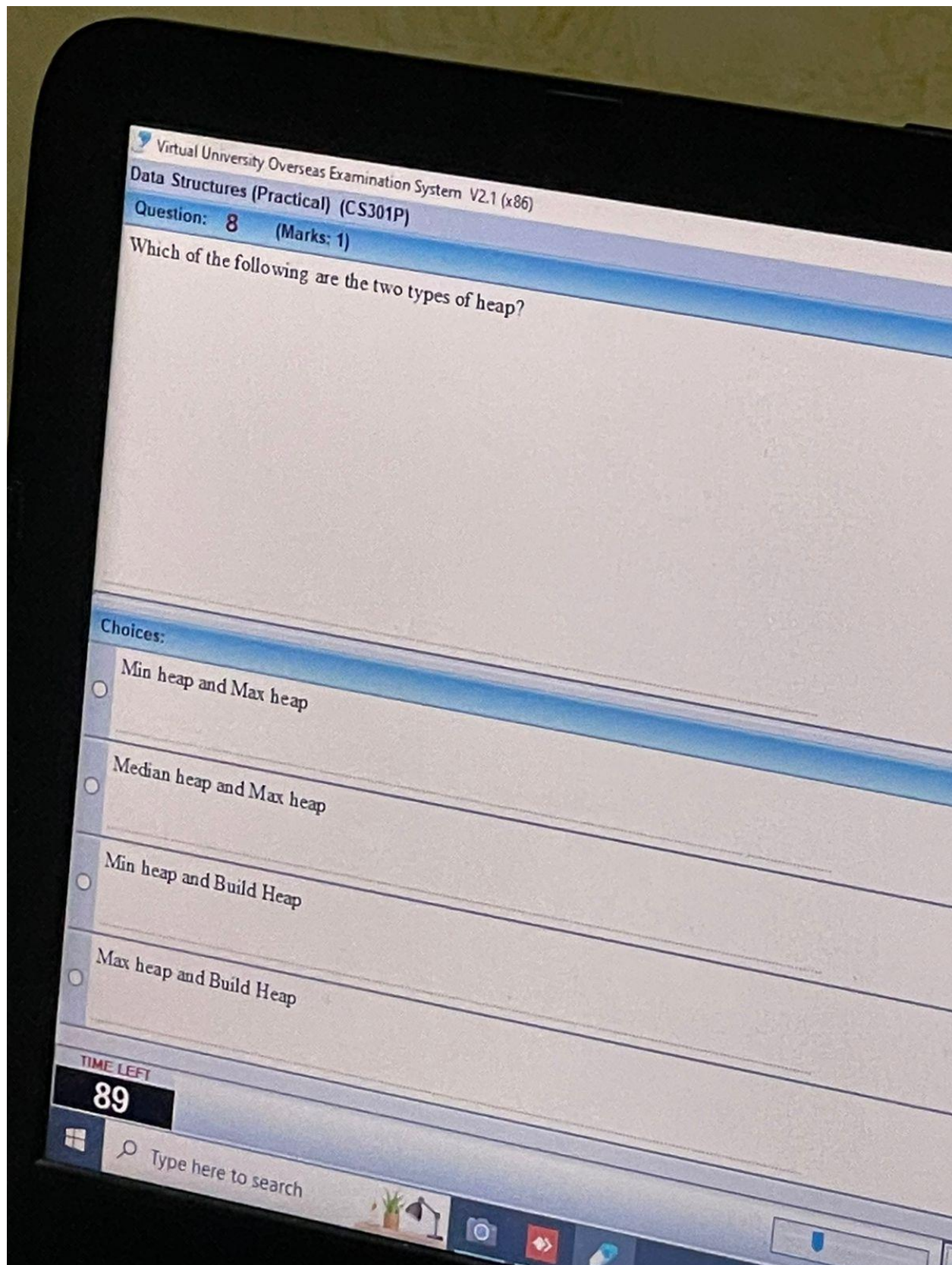
Choices:

- ☐ Single left Rotation
- ☐ Single right Rotation
- ☐ Left Right Rotation
- ☐ Right left Rotation

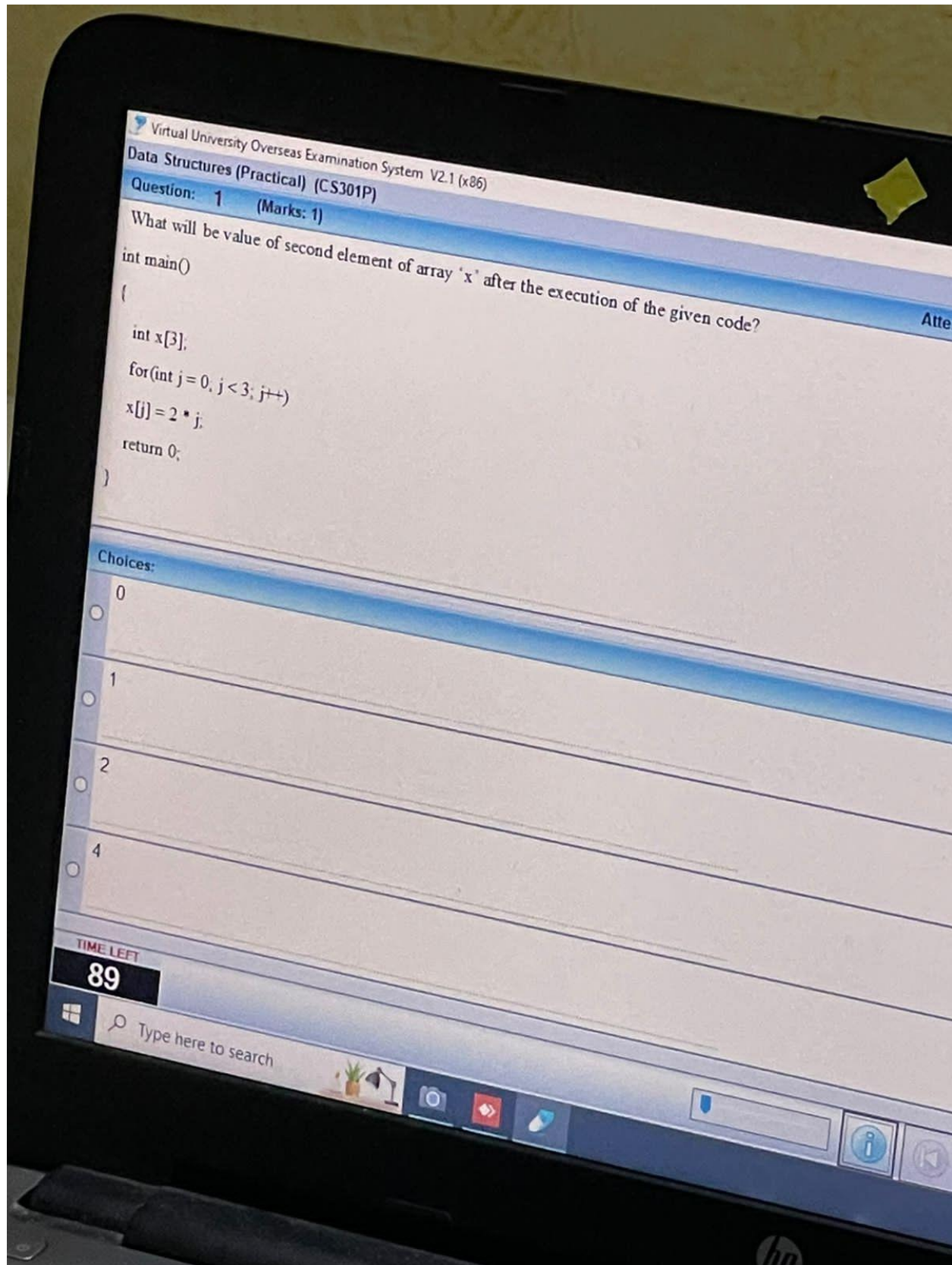
TIME LEFT
89

Type here to search

CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR

Virtual University Overseas Examination System V2.1 (x86)

Data Structures (Practical) (CS301P)

Question: 29 (Marks: 5)

There are many types of trees, consider the binary tree given below and write which type of tree's requirement it is fulfilling. You need to write only tree type names.

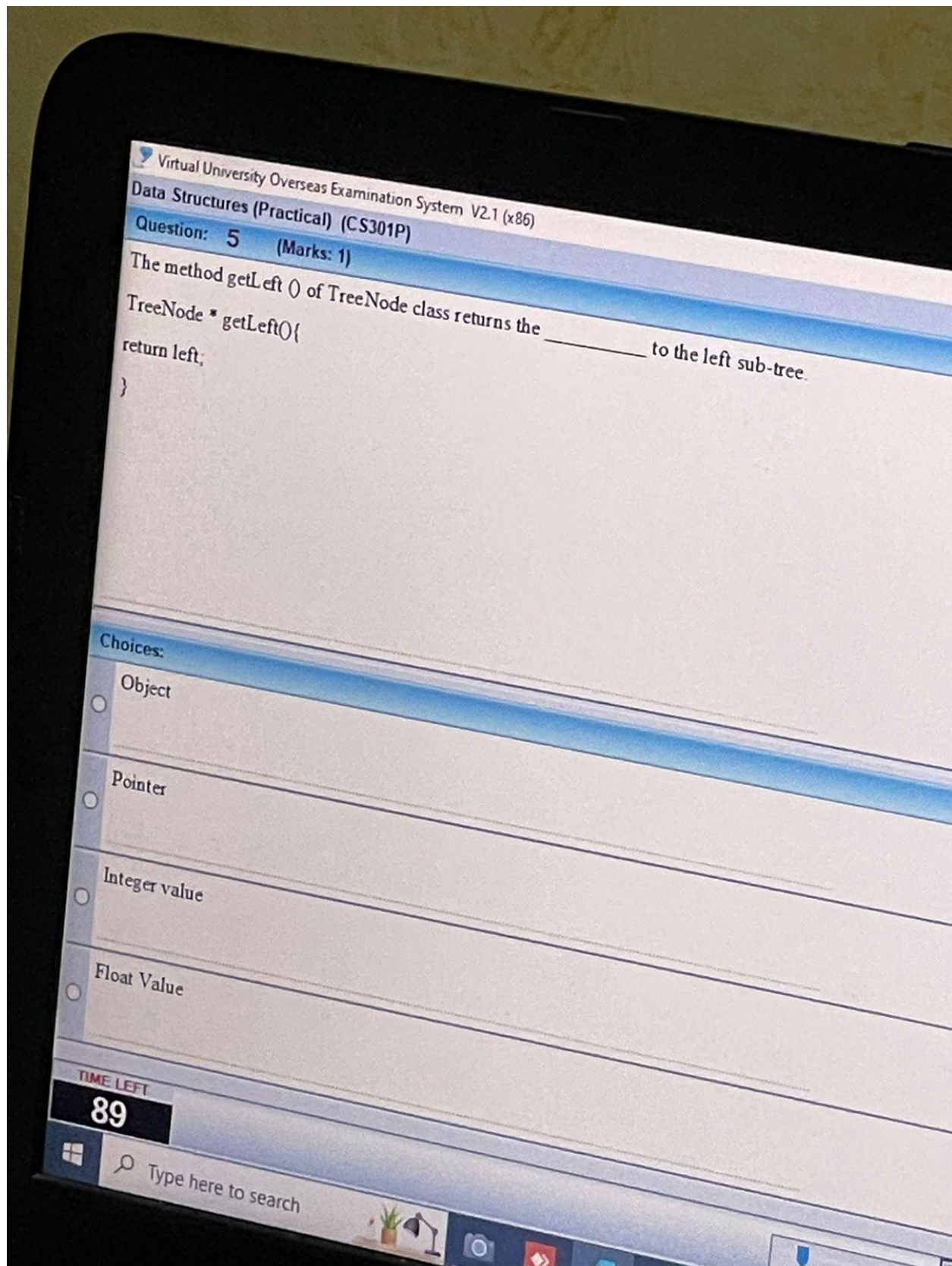
```
graph TD; 60((60)) --- 45((45)); 60 --- 70((70)); 45 --- 33((33)); 45 --- 50((50)); 70 --- 65((65)); 70 --- 77((77))
```

Answer:

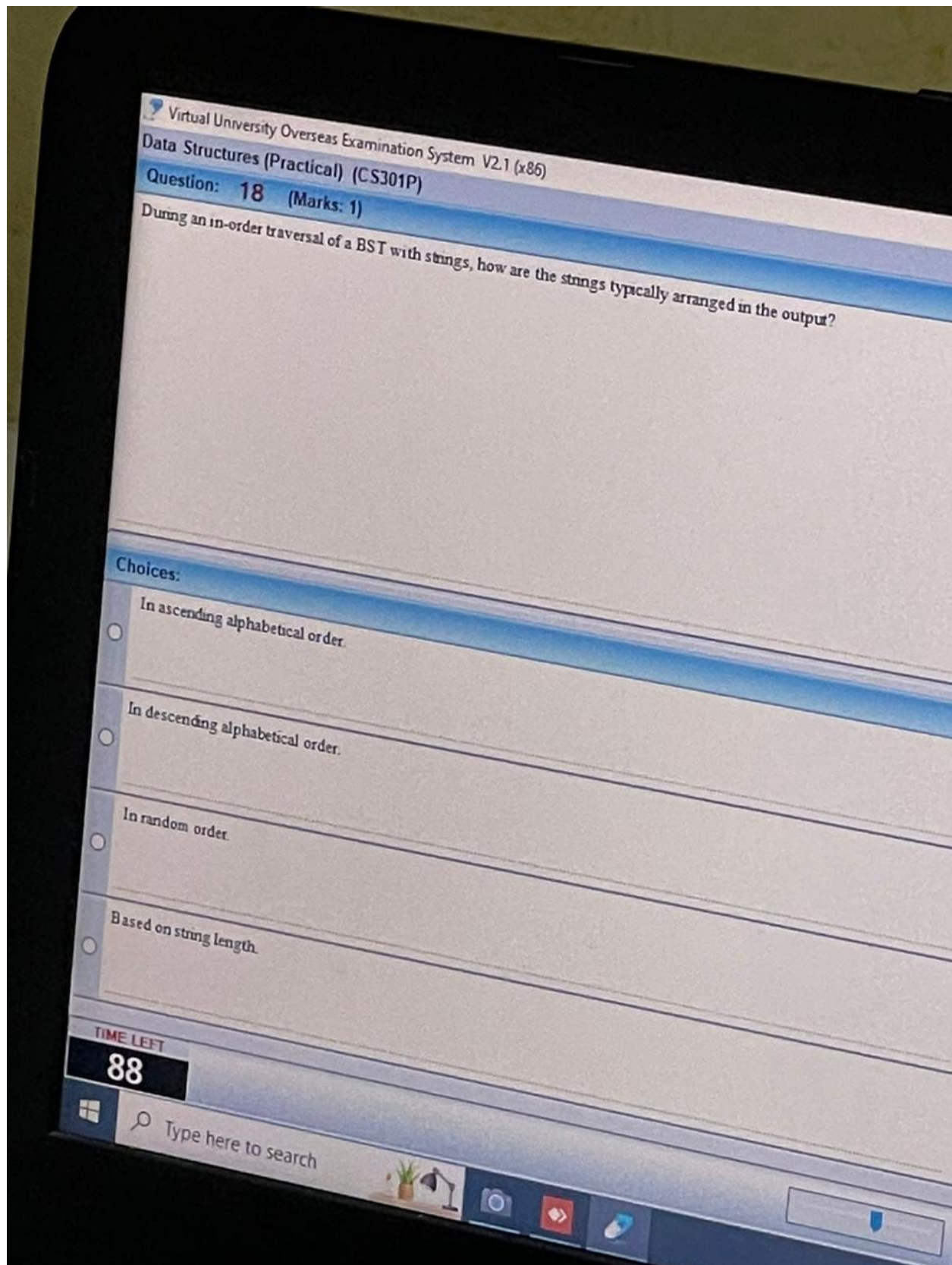
TIME LEFT: 90

Type here to search

CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR



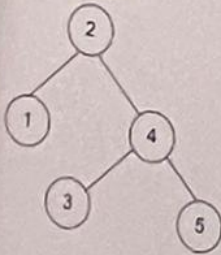
CS301P FINAL TERM BY IQRA AZHAR

Virtual University Overseas Examination System V2.1 (x86)

Data Structures (Practical) (CS301P)

Question: 11 (Marks: 1)

What will be the inorder traversal of the given AVL tree?



```
graph TD; 2((2)) --- 1((1)); 2 --- 4((4)); 4 --- 3((3)); 4 --- 5((5));
```

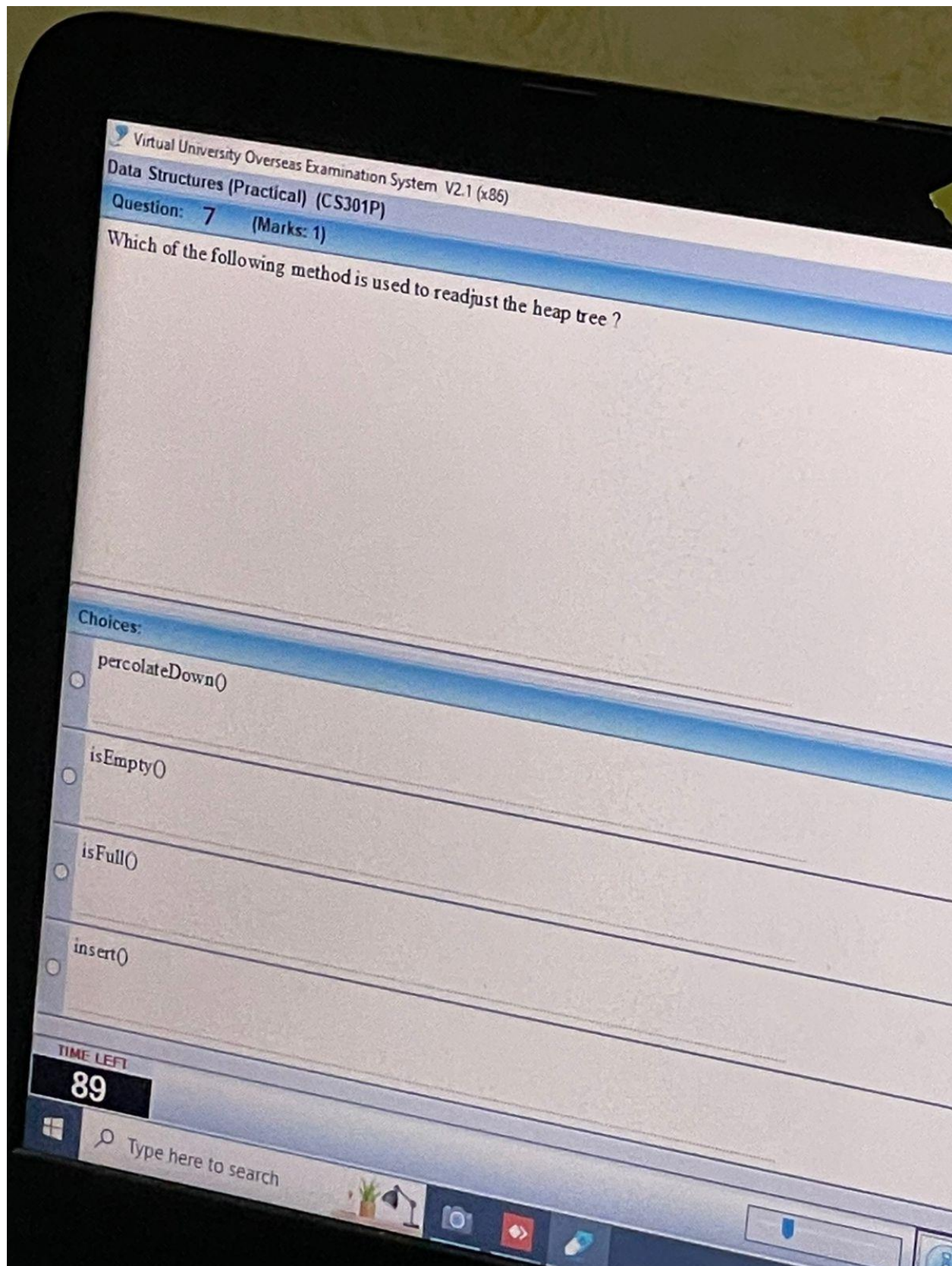
Choices:

- ☐ 5 4 3 2 1
- ☐ 1 3 5 4 2
- ☐ 2 1 4 3 5
- ☐ 1 2 3 4 5

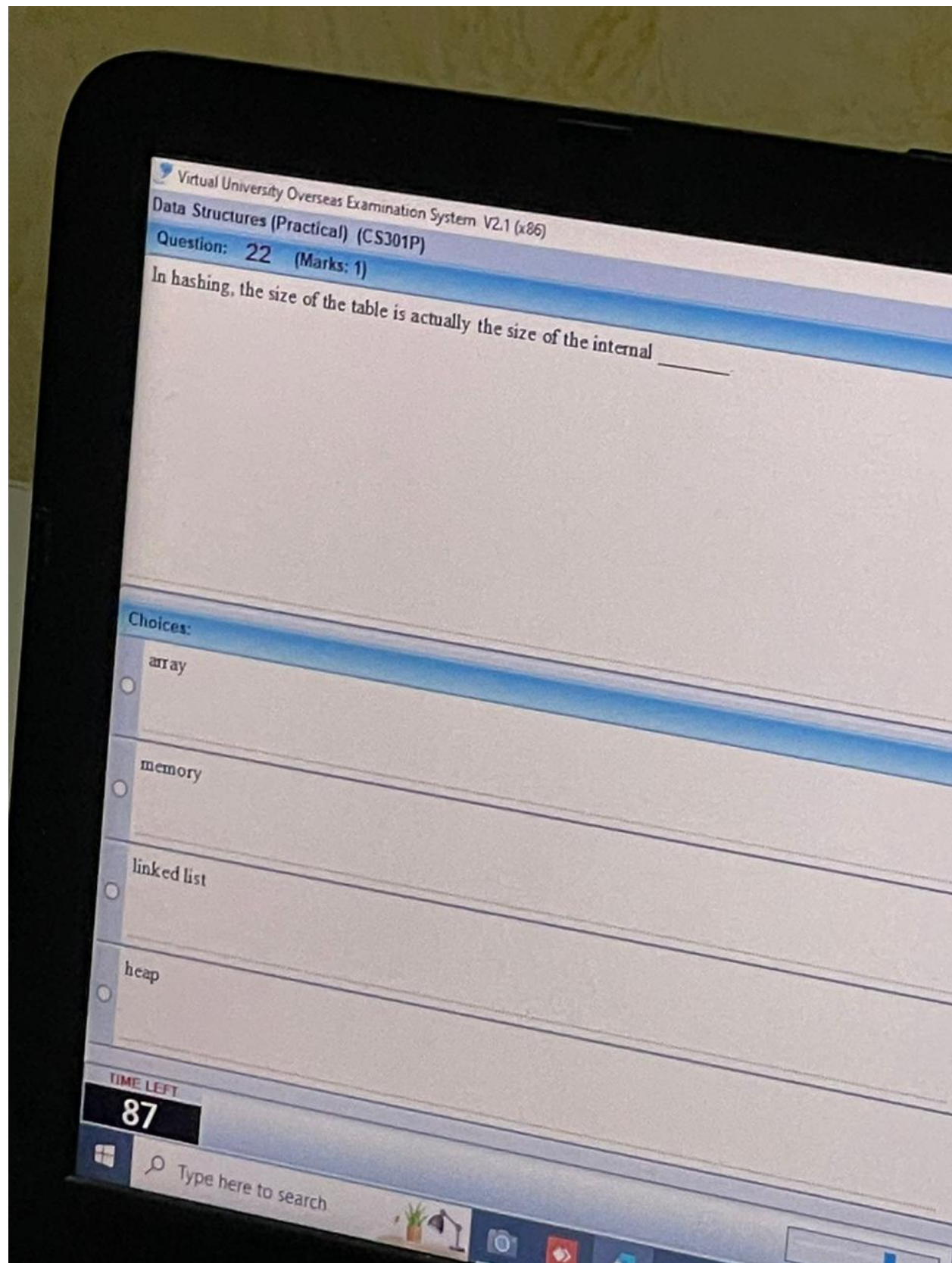
TIME LEFT
88

Type here to search

CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR

Virtual University Overseas Examination System V2.1 (x86)

Data Structures (Practical) Question 29 (Marks: 5)

Attempt

There are many types of trees, consider the binary tree given below and write which type of tree's requirement it is fulfilling. You need to write only tree type names.

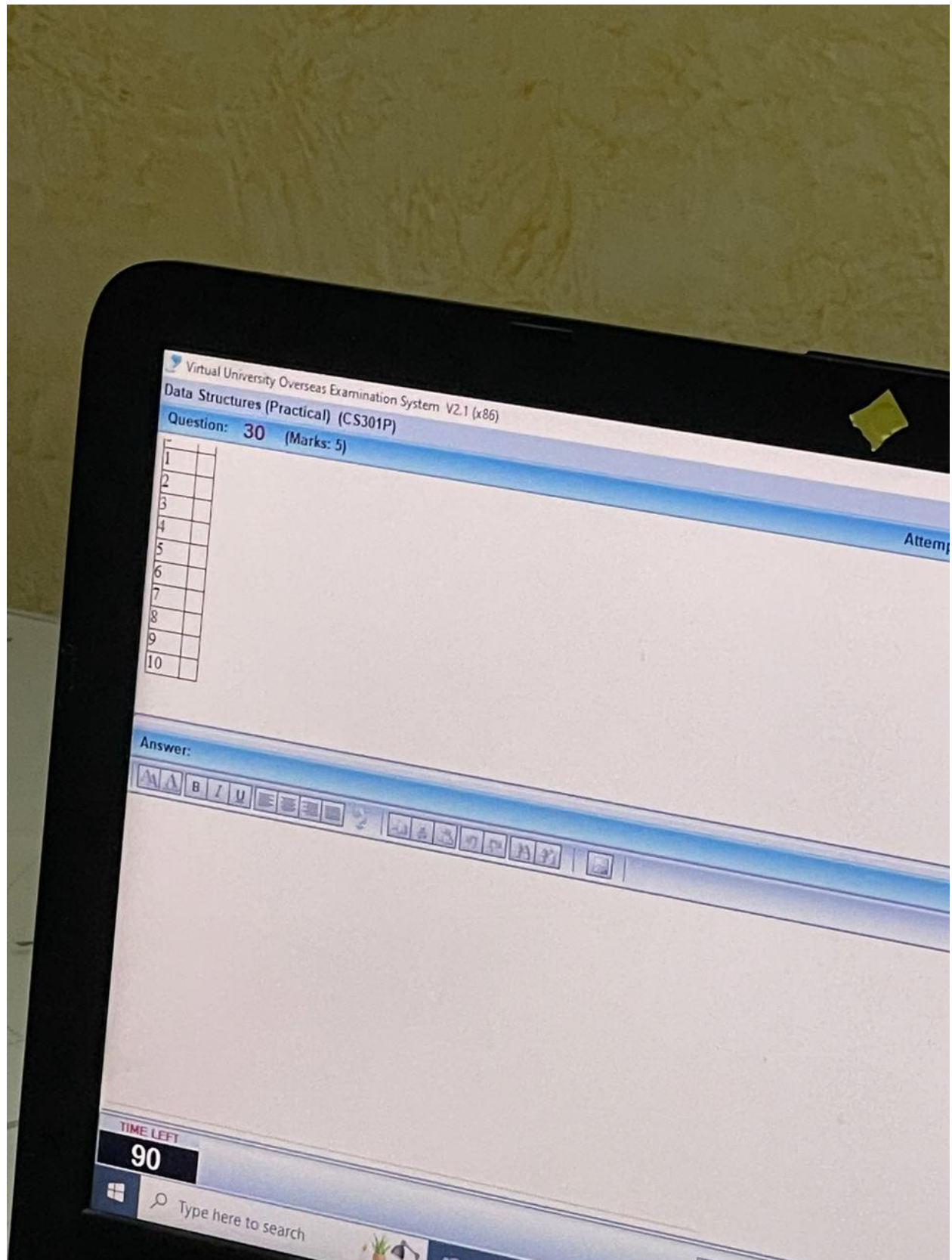
```
graph TD; 60((60)) --> 45((45)); 60 --> 70((70)); 45 --> 33((33)); 45 --> 50((50)); 70 --> 65((65)); 70 --> 77((77));
```

Answer:

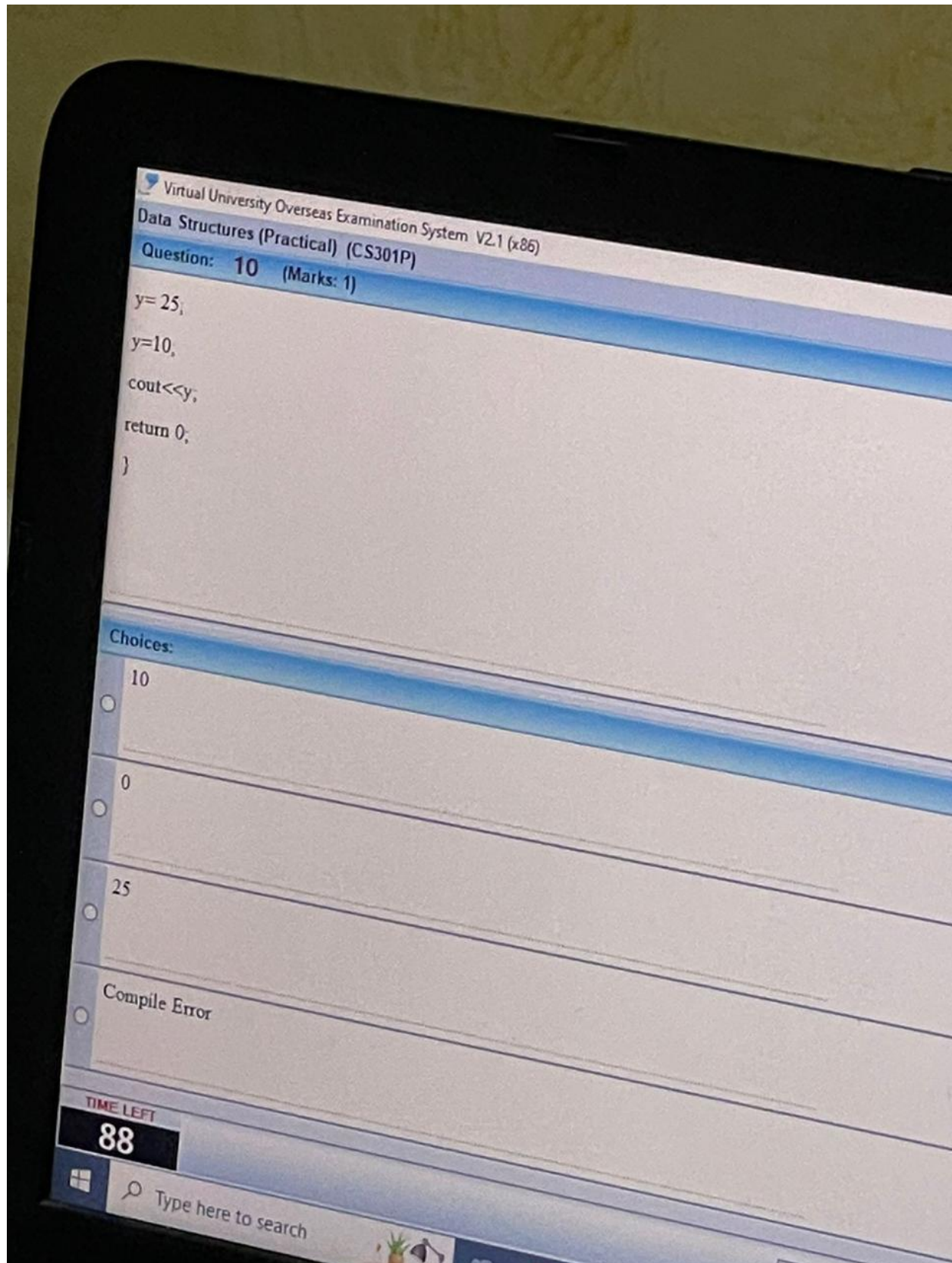
TIME LEFT 90

Type here to search

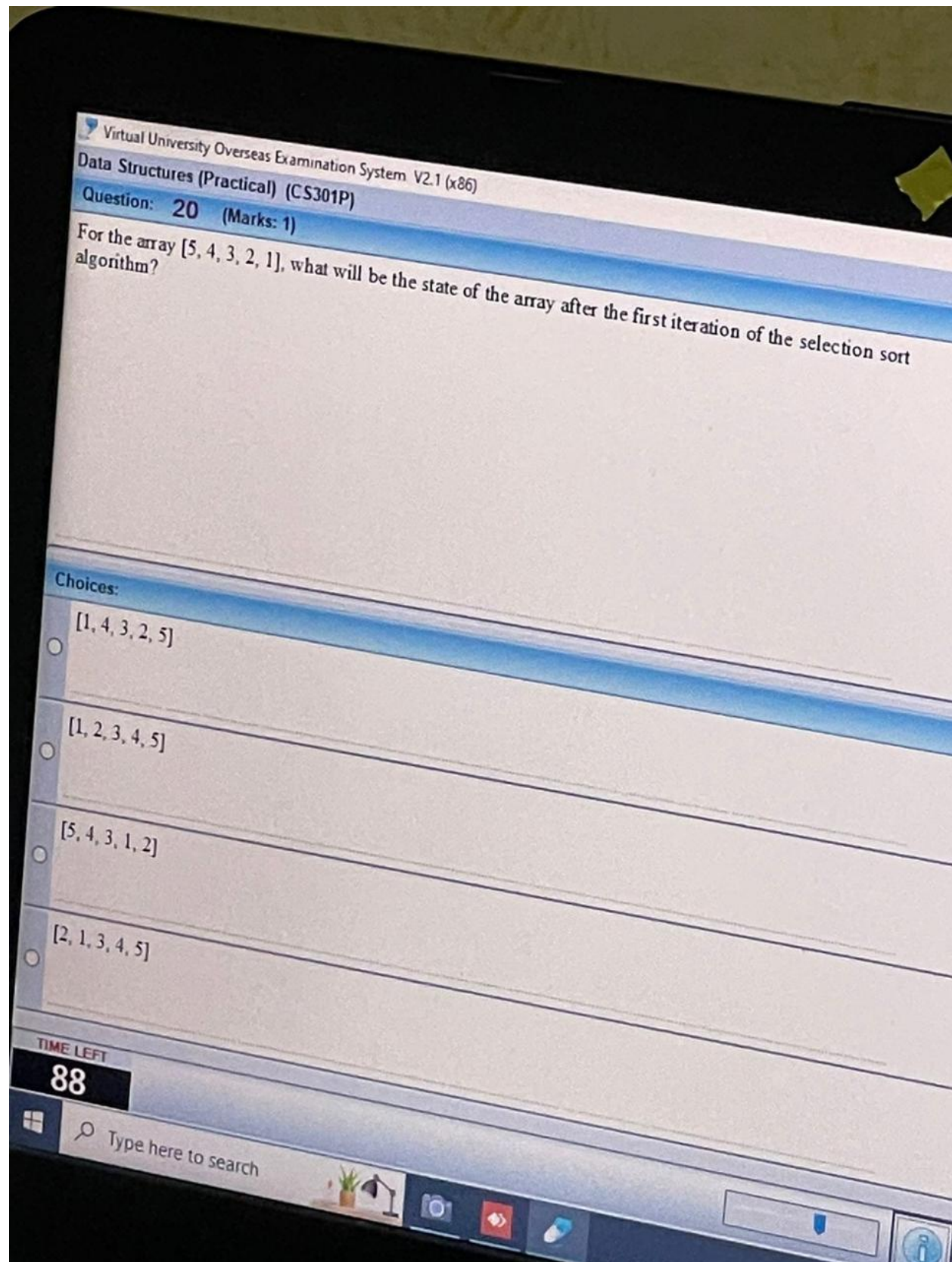
CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR

Virtual University Overseas Examination System V2.1 (x86)

Data Structures (Practical) (CS301P)

Question: 19 (Marks: 1)

How many rotations are required to balance the following AVL tree?

```
graph TD; 45((45)) --- 42((42)); 45 --- 48((48)); 48 --- 54((54)); 54 --- 65((65));
```

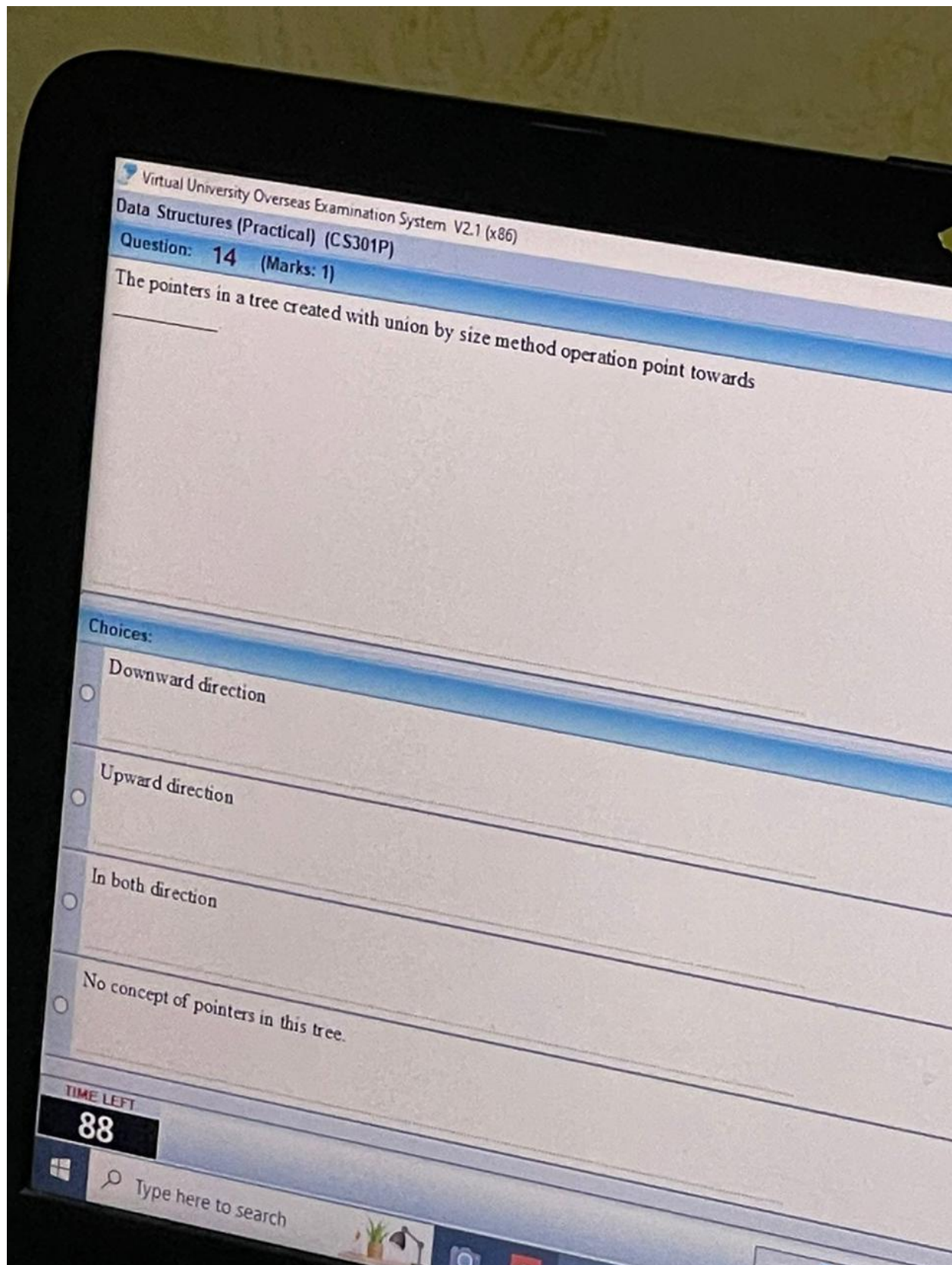
Choices:

- ☐ 0
- ☐ 1
- ☐ 2
- ☐ 3

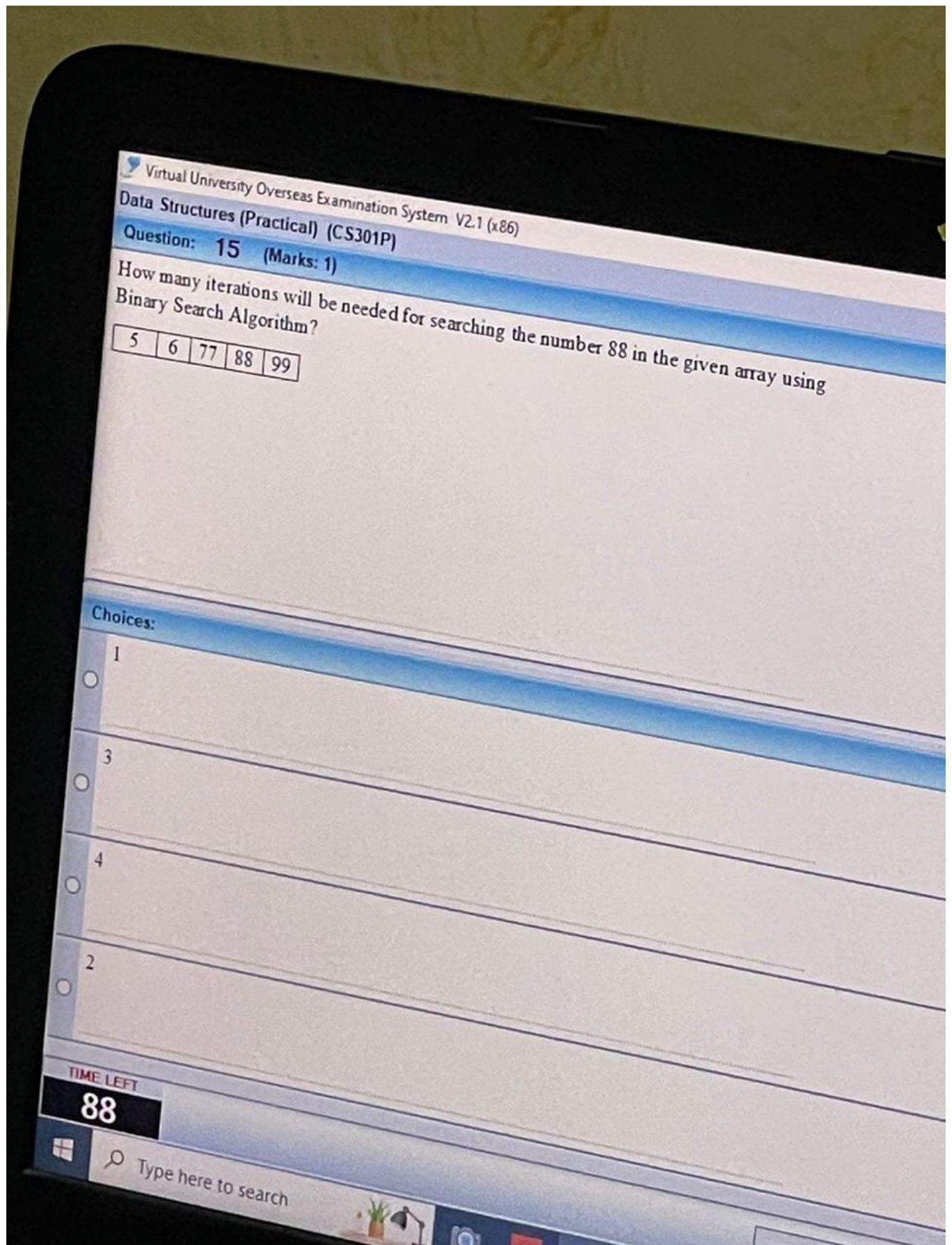
TIME LEFT
88

Type here to search

CS301P FINAL TERM BY IQRA AZHAR



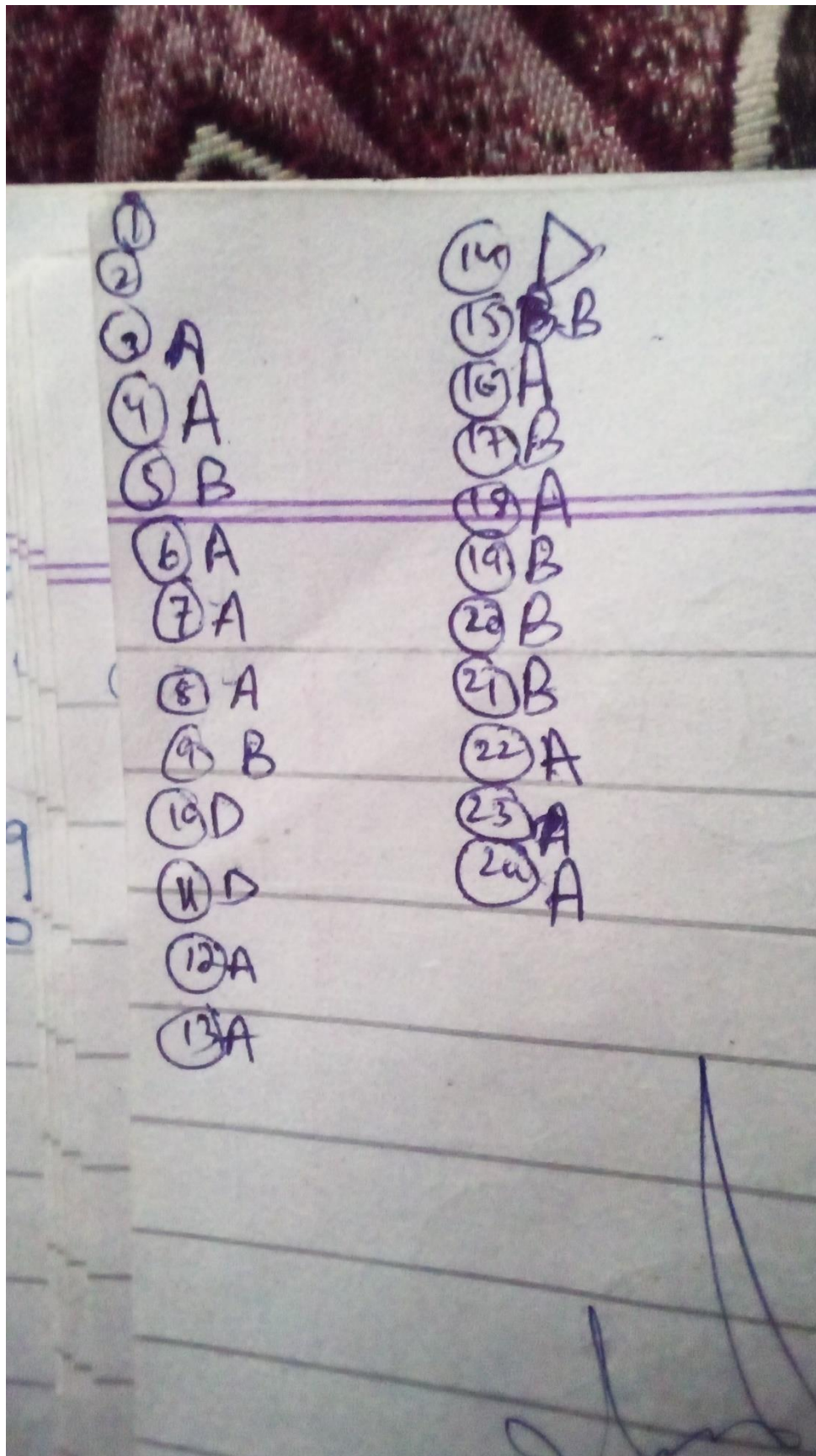
CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR

Solution must confirm the answer

CS301P FINAL TERM BY IQRA AZHAR



CS301P FINAL TERM BY IQRA AZHAR

Q25

```
#include <iostream>
```

```
using namespace std;
```

```
class Stack {
```

```
private:
```

```
    int size;
```

```
    int current;
```

```
    int A[10];
```

```
public:
```

```
    // Constructor
```

```
    Stack(int size = 10, int current = -1) : size(size), current(current) {}
```

```
// Method to check if the stack is full
```

```
    bool isFull() {
```

```
        return current == size - 1;
```

```
    }
```

```
// Method to check if the stack is empty
```

```
    bool isEmpty() {
```

```
        return current == -1;
```

```
    }
```

```
// Method to pop element from the stack
```

```
    int pop() {
```

```
        if (!isEmpty()) {
```

```
            return A[current--];
```

CS301P FINAL TERM BY IQRA AZHAR

```
    } else {

        cout << "Stack underflow! Cannot pop from an empty stack." << endl;

        return -1; // Assuming -1 indicates an error or empty stack

    }

}

// Method to push element onto the stack

void push(int x) {

    if (!isFull()) {

        A[++current] = x;

    } else {

        cout << "Stack overflow! Cannot push onto a full stack." << endl;

    }

}

// Method to get the top element of the stack

int top() {

    if (!isEmpty()) {

        return A[current];

    } else {

        cout << "Stack is empty! Cannot retrieve top element." << endl;

        return -1; // Assuming -1 indicates an error or empty stack

    }

}

};
```

CS301P FINAL TERM BY IQRA AZHAR

```
int main() {  
    // Example usage  
  
    Stack myStack(10);  
  
    cout << "Is the stack full? " << (myStack.isFull() ? "Yes" : "No") << endl;  
  
    cout << "Is the stack empty? " << (myStack.isEmpty() ? "Yes" : "No") << endl;  
  
    return 0;  
}
```

Q 26

```
#include <iostream>  
  
using namespace std;  
  
// Node class for the linked list  
  
class Node {  
  
public:  
  
    int data;  
  
    Node* next;  
  
    Node(int val) : data(val), next(nullptr) {}  
  
};  
  
// Queue class implemented using a linked list  
  
class Queue {  
  
private:  
  
    Node* front;  
  
    Node* rear;
```

CS301P FINAL TERM BY IQRA AZHAR

public:

// Constructor

Queue() : front(nullptr), rear(nullptr) {}

// Method to remove an element from the front of the queue (dequeue)

int dequeue() {

if (isEmpty()) {

cout << "Queue is empty. Cannot dequeue." << endl;

return -1; // Assuming -1 indicates an error or empty queue

}

int removedData = front->data;

Node* temp = front;

front = front->next;

delete temp;

if (front == nullptr) {

rear = nullptr; // If queue becomes empty after dequeue

}

return removedData;

}

// Method to check if the queue is empty

bool isEmpty() {

return front == nullptr;

}

};

CS301P FINAL TERM BY IQRA AZHAR

```
int main() {  
  
    // Example usage  
  
    Queue myQueue;  
  
    // Dequeue from an empty queue  
  
    cout << "Dequeuing from an empty queue: " << myQueue.dequeue() << endl;  
  
    // Enqueue some elements  
  
    for (int i = 1; i <= 5; ++i) {  
  
        myQueue.enqueue(i * 10);  
  
    }  
  
    // Dequeue elements and display  
  
    cout << "Dequeuing elements: ";  
  
    while (!myQueue.isEmpty()) {  
  
        cout << myQueue.dequeue() << " ";  
  
    }  
  
    cout << endl;  
  
    return 0;  
  
}
```

Q28

```
#include <iostream>  
using namespace std;
```

```
template <class eType>  
class Heap {  
private:  
    int capacity;  
    int size;  
    eType *elements;
```

CS301P FINAL TERM BY IQRA AZHAR

public:

```
// Constructor
Heap(int capacity) {
    this->capacity = capacity;
    this->size = 0;
    this->elements = new eType[capacity];
}

// Destructor
~Heap() {
    delete[] elements;
}

// Method to check if the heap is empty
bool isEmpty() {
    return size == 0;
}

// Method to check if the heap is full
bool isFull() {
    return size == capacity;
}
};

int main() {
    // Example usage
    Heap<int> myHeap(10);

    cout << "Is the heap empty? " << (myHeap.isEmpty() ? "Yes" : "No") << endl;
    cout << "Is the heap full? " << (myHeap.isFull() ? "Yes" : "No") << endl;

    return 0;
}
Q29
```

he tree in the image is a complete binary tree.

A complete binary tree is a binary tree in which every level, except possibly the last, is completely filled, and all the nodes are as far left as possible. This means that each level of the tree has the maximum possible number of nodes. The last level may be filled with nodes from the left side to the right, but there can be no gaps in the level.

The tree in the image meets all of these criteria. It has four levels, and the first three levels are all completely filled. The last level has three nodes, but they are all as far left as possible.

Here are some of the properties of complete binary trees:

CS301P FINAL TERM BY IQRA AZHAR

The height of a complete binary tree is $\log_2(n) + 1$, where n is the number of nodes in the tree.

The number of nodes in a complete binary tree with height h is $2^{(h+1)} - 1$.

The in-order traversal of a complete binary tree visits the nodes in increasing order of value.

The pre-order traversal of a complete binary tree visits the root node first, then the left subtree, and then the right subtree.

The post-order traversal of a complete binary tree visits the left subtree first, then the right subtree, and then the root node.

Q30

Index Value

0

1

2

3

4

5 5

6

7

8

9

10

Inserting 12:

Index Value

0

CS301P FINAL TERM BY IQRA AZHAR

1

2

3 12

4

5 5

6

7

8

9

10

Inserting 19:

Index Value

0

1

2

3 12

4 19

5 5

6

7

8

CS301P FINAL TERM BY IQRA AZHAR

9

10

Inserting 26:

Index Value

0

1

2

3 12

4 19

5 5

6 26

7

8

9

10

So, the final table after inserting 5, 12, 19, and 26 using linear probing resolves collisions as shown